系统概述

在一个高并发的事务型系统中，当写事务占的比例相对读事务相对较小时，可以借助 DM7 的主备系统备库可读的特点，将读事务转移到备库执行，减少单节点的并发压力，通过增加备库节点资源，提高系统的并发能力，增强系统性能。

DM7 提供一种独具创新的主备方案，读写分离主备系统，该系统可通过客户端来实现读写事务的自动分离，读事务在备库执行，写事务在主库执行，减轻主库的负载。备库可以配置多个，理论上说，在允许的范围内，备库配置的越多，更能分担主库的压力，系统整体并发效率越高。

目前该系统：

- 限制最多支持 8 个即时备库
- 支持 JDBC、DPI、DCI、ODBC、Provider 等多种接口。

即时归档

DM7 支持多种归档类型，本地归档、实时归档、同步归档、异步归档等类型。为了实现读写分离，新增一种即时归档（Timely archive）类型，以区别实时归档。

实时归档是实时发送日志到备库，备库收到日志不会等待日志 APPLY 完成，立即响应给主库，主库收到响应后才刷本地日志。

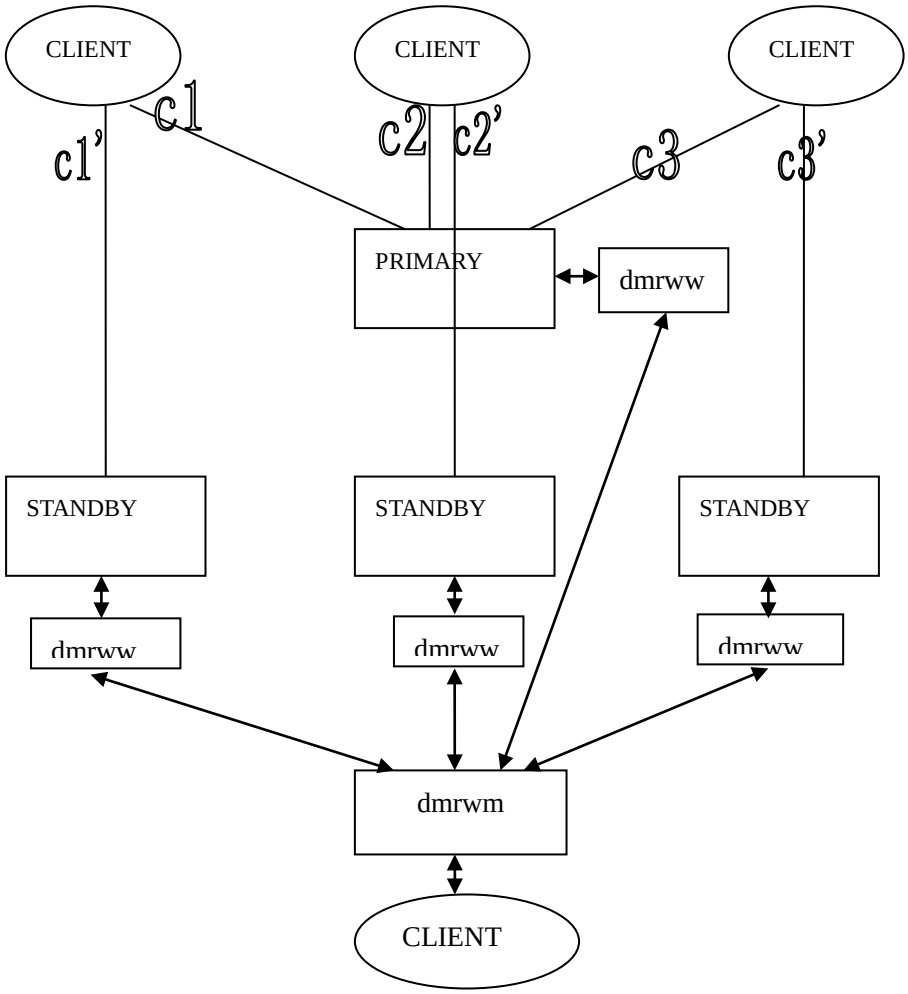
即时归档的归档方式是先刷本地日志，然后发送到备库，并且等待备库 APPLY 完成，之所以要这么做，原因有 2 个:

首先,在主库发送完日志到备库后刷日志前主库崩溃的情况下,不能让备库多一段日志，确保主库数据是完整且最新的。

其次，需要保证主备库的数据一致性，避免从备库读的数据到主库进行更新时数据已经不一致了。

在某些不需要保证严格一致性的情况下，可以通过 dmarch.ini 中的 ARCH_WAIT_APPLY 配置项，来配置不需要等待备库重做完日志的主备系统，类似于实时归档，可以进一步提高系统性能。

系统架构



如上图：

系统架构中包含一个主库实例，多个备库实例，带双向箭头的为内网传输线路，带标注线的为外网应用传输线路。

- **读写分离流程**

1） 用户登录后，客户端首先连接到主库，主库根据即时归档的配置，获取一个有效的备库信息，并返回给客户端。

2） 客户端根据主库返回的备库 IP 和端口，建立与该备库的连接

3） 客户端执行语句时先在备库上执行，如果是只读事务，则只在备库上执行

4） 如果报试图在备库模式下修改数据等错误，则说明该事务是写事务，继续转移到主库上执行

5） 一旦主库上执行的写事务提交，则下次继续从备库开始执行

6） 为了实现负载均衡，防止出现读事务过多占用备库资源、主库空闲的情况，客户端采用一定的算法进行均衡，主库上也会执行一部分读事务

- **归档失败处理**

  如果主库发送归档失败，需要进行如下处理：

  1） 设置对应备库的即时归档失效

  2） 主库上强制断开所有连接了该备库对应的在主库上的会话，客户端检测到主库断开后，也会强制断开备库的连接，防止数据读写的不一致

  3） 客户端检测到会话上的主备库任何一个连接断开，都会断开另外一个连接，重连时先连接主库，由主库返回其可连接的备库

- **dmrww 作用**

使用 UDP 或 TCP 协议和服务器实例及 dmrwm 工具通信，主库 dmrww 可以接收所有备库 dmrww 的广播消息。

dmrww 可以配置为自动或非自动守护模式,这两种模式下都会自动 open 所守护的实例，在自动模式下，主库 rww 可以自动恢复故障备库，故障主库的恢复需要在 dmrwm 端通过 recover 命令手动执行。

1） 接收实例消息，判断满足 open 条件则自动 open 实例

2） 广播实例状态给 dmrwm

3） 接收 dmrwm 消息，并转发给实例

4） 主库 dmrww 接收并处理备库 dmrww 的广播消息，自动模式下，备库满足恢复条件的话，会自动恢复故障备库

5） 检测到系统中存在多个 primary&open 状态的实例时，强制对这些实例执行 shutdown 操作。

- **dmrwm 作用**

该工具主要用来管理读写分离系统，允许用户通过输入命令的方式来执行主备库的启动、主库切换、备库恢复等操作。

1） 通过命令方式控制系统的启动和退出

2） 提供 LOGIN/LOGOUT 命令，登录/退出 dmrwm

3） 通过命令方式控制 dmrww 监控功能的打开/关闭

3） 接收 dmrww 消息，监控主备库状态，出现故障或状态变化时有提示信息

4） 提供切换命令，主备库正常时，可以执行主备库的切换

5） 提供选择命令，主库故障时，可以选出能够接管主库的备库

6） 提供接管命令，主库故障时，可以使用指定的备库接管故障主库

7） 提供恢复命令，主备库故障时，可以执行恢复操作

8） 提供 TIP 命令，可以检测系统当前运行状况

系统适用性

读写分离系统适用于读写比例较大的应用系统，读写比例越大对系统性能提升越明显。对于一般简单应用业务来讲，若需要使用读写分离系统提升系统性能，建议读写比例要大于1:1。

系统事务特征

读写分离系统实现读写分离是按事务来进行分发的。若一个事务全为读操作，则全部在备库上执行；若一个事务全为写操作，则全部在主库上执行；若一个事务中既有读又有写，备库会将写操作返回给主库执行，该事务中从写操作开始以后所有操作均在主库上执行，以保证事务的一致性。既有读又有写的事务处理过程相对于纯读、纯写的事务处理过程要耗费较多的时间。因此，应用程序对事务进行设计时，尽量保证事务为纯读或纯写。若一个事务中必须要有读、有写，则应当遵循以下原则：

- ✓ 读操作尽量放在写操作之前，用备库可读的特点来分摊系统压力；
- ✓ 在业务允许条件下，可将该事务拆分为一个纯读和一个纯写的事务；
- ✓ 写操作后尽量不要有读操作，避免压力集中在主库，影响系统性能。

系统配置

配置文件

读写分离系统涉及到的配置文件有dm.ini、dmmal.ini、dmarch.ini、dmrww.ini、dmrwm.ini。

dm.ini 和 dmmal.ini 的配置和守护进程中的配置一致。

dmrww.ini 中配置实例相关的信息及端口。

dmrwm.ini 中配置监视器的端口、组号等信息。

dmarch.ini 中除了配置本地归档外，需要配置到其他所有实例的即时归档，以便即时备库切换为主库后，可以正常向其他实例发送即时归档。另外 ARCH_WAIT_APPLY 参数值都要一致。

dmrww.ini

表 1 dmrww.ini 配置项

配置项	配置含义

[GROUP_NAME]	守护进程组名
RWW_UDP_TYPE	UDP 通信方式，0 表示广播方式，1 表示组播方式，默认为 0，TCP 方式下不需要配置此项。
RWW_UDP_TTL	UDP 组播方式下的数据包生存期，数据包传送时最大可经过的路由器个数，取值范围为 1 到 128，默认为 3，TCP 方式下不需要配置此项。
RWW_BRO_ADDR	UDP 广播或组播地址，TCP 方式下不需要配置此项： 广播地址根据 dmmal.ini 中的 MAL_HOST 配置项计算得出。 组播地址取值范围为(224.0.1.0-- 238.255.255.255)，同一套读写分离系统中需要取相同的组播地址。
RWW_UDP_PORT	守护进程 UDP 广播端口，和 dm.ini 的 DW_UDP_PORT 配置项，以及 dmrwm.ini 的 RWM_UDP_PORT 配置项保持一致，守护进程通过该端口接收本地实例消息，在 TCP 方式下也需要配置，用于接收服务器发送的 UDP 消息。
RWW_INST_ERROR_TIME	本地数据库故障认定时间，取值范围为（3s~50000s），缺省 5 秒没有收到数据库发送的状态信息，即认定其监控的数据库出现故障
RWW_FAR_ERROR_TIME	远程守护进程故障认定时间，取值范围为（3s~50000s），缺省 5 秒没有收到远程守护进程广播的 UDP 消息，即认定远程的 dmrww 故障。
RWW_MODE	守护进程的守护模式，0 表示非自动模式，1 表示自动模式，默认为自动模式，自动模式下 dmrww 可以自动恢复故障备库。
RWW_TCP_PORT	TCP 通信方式下，守护进程的 TCP 监听端口，用于监听远程守护进程和监视器的连接请求，同一套读写分离主备系统中，该配置项需要保持一致，监视器的 RWM_RWW_TCP_PORT 配置项也要一致
RWW_MAL_INI	TCP 通信方式下指定 dmmal.ini 文件的绝对路径，用于查找远程守护进程所在 IP 地址
RWW_ARCH_INI	TCP 通信方式下指定 dmarch.ini 文件的绝对路径
RWW_ARCH_TYPE	TCP 通信方式下指定归档类型，即时归档方式下配置为 TIMELY
INST_OGUID	唯一标识一个读写分离系统，不能和其他读写分离系统的 oguid 相同，同一个读写分离系统中的所有数据库、守护进程和监视器，都必须配置相同的 OGUID 值，取值范围为 0-2147483647
INST_NAME	守护进程监控的数据库实例名，与 dm.ini 的 INSTANCE_NAME 配置项保持一致
INST_UDP_PORT	数据库 UDP 端口，守护进程使用这个端口向数据库发送 UDP 消息，与 dm.ini 的 INST_UDP_PORT 配置项保持一致，TCP 方式下也需要配置此项。
INST_STARTUP_MODE	数据库启动模式，0 表示以服务方式启动数据库，1 表示以命令行方式启动数据库
INST_STARTUP_CMD	数据库启动命令，可参考数据守护的配置说明
INST_AUTO_RESTART	实例故障后，守护进程是否自动重启实例（取值 0、1，默认自动

	重启，值为 1）

参数说明：

RWW_MODE：

自动或非自动模式下，dmrww 检查本地实例满足 open 条件时都会执行自动 open 操作。

自动模式下，主库正常时，dmrww 检测到故障备库后会执行自动恢复，非自动模式下需要在 dmrwm 端手动执行 recover 命令恢复备库。

被接管后的故障主库的恢复，都需要通过 dmrwm 的 recover 命令手动恢复。

注意：

对于备库来说，这里的恢复是指主备库之间有 lsn 差值，需要主库同步归档日志到备库重做，直到备库 lsn 和主库一致，然后修改备库即时归档为 VALID 状态的过程。

对于主库来说，这里的恢复是指将主库作为备库重加入的过程，包括补齐和新主库之间的 lsn 差值。

实例故障后，如果 INST_AUTO_RESTART 配置为 1，会将实例重新拉起，但如果主备库之间存在 lsn 差值，在自动模式下，dmrww 会自动执行这个恢复过程。

守护进程通信方式说明：

读写分离系统中，守护进程(dmrww)和本地实例、远程守护进程(dmrww)、监视器之间都有通信，其中守护进程和本地实例之间使用 UDP 方式定向发送消息，守护进程之间、守护进程和监视器之间支持 TCP 和 UDP 两种通信方式， 其中 UDP 方式支持广播和组播配置，广播方式需要守护系统部署在同一个局域网内，组播方式和 TCP 方式同时支持单个局域网和跨网段部署。

TCP 及 UDP 相关的配置参数请参考 dmrww.ini 配置表格说明，如果同时配置有 TCP 和 UDP 参数，则选择 TCP 方式进行通信。守护进程之间、以及守护进程和监视器之间的通信方式需要保持一致，都使用 TCP 或都使用 UDP 方式，否则读写分离系统无法正常使用。

组播配置说明：

系统支持 UDP 组播配置方式，允许跨网段搭建，UDP 组播方式条件比较严格，注意事项如下：

1.单个局域网内支持组播方式配置，不需要支持组播协议。

2.跨网段配置时，连接不同网段的路由器、交换机等设备必须能够支持组播协议，并打开相关组播协议。

3.组播组内的主库如果没有组播路由，则需要添加组播路由到路由表中，且需要保证添加组播路由时绑定的网卡设备 IP 地址协议和组内其他主库一致，都使用 IPV4 或 IPV6 协议。

添加组播路由命令（不同操作系统命令格式可能不同，需要根据实际情况调整）：

a. 操作系统 Linux localhost.144 2.6.32-220.el6.x86_64，添加路由命令：

```
route add -net 224.0.0.0 netmask 240.0.0.0 dev eth0
```

该命令将 D 类组播地址 224.0.0.0 加入到路由表中，子网掩码 240.0.0.0，绑定网卡设备 eth0，即组播消息通过设备 eth0 发送和接收，同一个多播组中绑定的设备支持的 IP 协议需要一致。

此处添加的是动态路由，系统重启后需要重新添加，可以将此命令添加到/etc/rc.local 文件，系统启动后自动执行。

b. 操作系统 AIX 7.1-64，添加路由命令：

```
route add -net 224.0.0.0 192.168.0.196 -if en0
```

该命令将 D 类组播地址 224.0.0.0 加入到路由表中，网关地址 192.168.0.196，绑定网卡设备 en0，即组播消息通过设备 en0 发送和接收。

此处添加的是动态路由，系统重启后需要重新添加，AIX 可以通过自带的 smit 工具，使用 smit mkroute 命令进入配置界面，增加为静态路由。

查看路由表命令：

netstat –rn

```
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
224.0.0.0       0.0.0.0         240.0.0.0       U         0 0          0    eth0
```

守护进程或监视器启动后，可以查看加入的多播组信息与网卡设备是否一致，下面示例中守护进程配置的组播地址为 234.1.1.1，与设备 eth0 关联。

查看多播组命令：

netstat -gn

```
IPv6/IPv4 Group Memberships
Interface       RefCnt Group
--------------- ------ ---------------------
eth0            1      234.1.1.1
```

dmrwm.ini

表 2 dmrwm.ini 配置项

配置项	配置含义
RWM_UDP_TYPE	UDP 通信方式，0 表示广播方式，1 表示组播方式，默认为 0
RWM_UDP_TTL	UDP 组播方式下的数据包生存期，数据包传送时最大可经过的路由器个数，取值范围为 1 到 128，默认为 3。
RWM_BRO_ADDR	UDP 广播或组播地址，与 dmrww.ini 中 RWW_BRO_ADDR 保持一致
RWM_UDP_PORT	UDP 广播端口，与 dmrww.ini 中 RWW_UDP_PORT 保持一致
RWM_INST_OGUID	数据守护唯一标识码，与 dmrww.ini 中的 INST_OGUID 保持一致
RWM_GROUP_NAME	守护进程组名，与 dmrww.ini 中的守护进程组名保持一致
RWM_CTL_PATH	控制文件路径，存放即时备库切换为主库时的相关信息
RWM_RWW_TCP_PORT	TCP 方式下，连接远程 dmrww 的 TCP 端口，和 dmrww.ini 中的 RWW_TCP_PORT 保持一致
RWM_RWW_IP	TCP 方式下，远程守护进程(dmrww)地址，和 dmmal.ini 中的 MAL_HOST 保持一致，该配置项最多允许配置 9 个（主库和 8 个即时备库），不允许有重复值。
RWM_LOG_PATH	日志文件路径，相关说明请参考下面的注意事项。 日志文件命名方式为 “dmrwm_端口号_年月日时分秒.log”，例如“dmrwm_55681_20150418230523.log”

RWM_LOG_INTERVAL	自动记录系统状态信息到日志文件的时间间隔，取值 0,1，或(5~3600)，单位秒，0 表示不记录任何日志，1 表示只记录监视器正常接收到的消息，5~3600 表示除了记录监视器正常接收消息外，每隔指定的间隔另外记录系统信息到日志文件中，默认值为 1。
RWM_LOG_FILE_SIZE	单个日志文件大小，范围 16~2048，单位为 M，默认值为 64，达到最大值后，会自动生成并切换到新的日志文件中，如果达到设定的总空间限制，会自动删除创建时间最早的日志文件。
RWM_LOG_SPACE_LIMIT	日志总空间大小，取值 0 或者 256~4096，单位为 M，默认值为 0，表示没有空间限制。

控制文件说明：

其中 dmrwm 的控制文件记录备库接管主库时的相关信息，包括新主库名、故障主库名以及备库接管主库时的 lsn 这三项。

在主库恢复做为备库重加入时，需要先查找控制文件，如果没有找到该主库的接管信息，则不允许加入。如果找到，则比较其 lsn 是否大于切换当时的 lsn，若大于则不允许主库加入，在满足条件加入成功后，控制文件中对应的接管记录会被删除。

如果发生多次备库接管，比如主库 A 故障，备库 B 接管为主库后又发生故障，又选用备库 C 接管，则控制文件中会依次记录 B、C 接管时的信息，在主库 A、B 重新加入时依次查找对应的记录，加入成功后删除相关记录。

该控制文件在执行接管命令的 dmrwm 所在机器上生成，生成路径为 RWM_CTL_PATH 配置的参数值，如果用户在多个机器上配置有监视器并执行过接管命令，则各自都会生成对应的控制文件，需要在对应机器上对被接管的主库执行恢复，否则会因为找不到接管信息不允许故障主库重加入。

也可以多个监视器共用同一个控制文件，注意在执行切换前，需要先拷贝到本地控制文件路径下，其他命令的执行则没有此限制。建议执行备库接管和主库恢复命令都在同一个监视器上执行，就不会受控制文件的限制。

接管说明：

在监视器上执行备库的接管命令时，一定要先确认是主库实例故障而不是内网网卡故障或其他网络问题。如果是内网网络问题，监视器收不到主库消息会误判主库出现故障，此时再执行备库接管，系统中就会出现双主库，老的主库继续在对外提供服务，并且双主库会同时向即时备库发送归档，这种情况下无法保证主备数据的正确性。

TCP 配置说明：

TCP 相关的配置参数为 RWM_RWW_TCP_PORT 和 RWM_RWW_IP，其中 RWM_RWW_IP 允许配置多个，指定所有远程守护进程的 IP 地址，不允许有重复值。

TCP 方式下，dmrwm.ini 配置表格中 UDP 相关的参数都可以不配置，如果同时配置有 UDP 和 TCP 两种通信方式，则使用 TCP 协议进行通信，会忽略 UDP 参数，未配置 TCP 参数才使用 UDP 协议，至少要配置一种方式。

守护进程(dmrww)和监视器之间的通信方式需要保持一致，都使用 TCP 或都使用 UDP 方式，否则读写分离系统无法正常使用。

日志相关配置说明：

dmrwm.exe 工具的日志文件路径以 dmrwm.ini 中配置的 RWM_LOG_PATH 为准，如果没有配置则使用 dmrwm.ini 配置文件所在的路径作为日志文件路径。

如果是通过调用接口方式来使用监视器的相关功能时，日志文件路径以 dmrwm_init 接口中指定的 log_path 为准，如果 log_path 参数值为 NULL 或空串，则以 dmrwm.ini 中的 RWM_LOG_PATH 为准，如果两个都没指定，则使用 dmrwm.ini 配置文件所在的路径

作为日志文件路径。

RWM_LOG_INTERVAL 配置为 0 时，不生成任何日志信息，RWM_LOG_PATH、RWM_LOG_FILE_SIZE 和 RWM_LOG_SPACE_LIMIT 的配置不起作用。

环境准备

硬件说明

部署 DM7 读写分离系统的基本硬件包括：

1) 主库

用于部署主数据库和守护进程，至少应具备两块网卡，一块接入公共网络，提供数据库服务，一块接入内部网络，传递 REDO 日志和 UDP 或 TCP 消息。

2) 备库

用于部署备数据库和守护进程，至少应具备两块网卡，一块接入公共网络，提供数据库服务，一块接入内部网络，接收 REDO 日志，发送并接收 UDP 或 TCP 消息，读写分离系统中最多可支持 8 个备库。

3) 监视机

用于部署监视器的机器，可同时配置多个监视器。

4) 内部网络

用于主、备库之间传输 REDO 日志，并同时用于守护进程和监视器广播状态与控制信息的网络环境。如果条件允许，最好配置多块物理网卡虚拟成一块网卡用于内部网络通信，提升内部网络的吞吐量和可靠性。内部网络通过交换机实现互联，最好能够提供冗余交换机本身，确保内部网络的可靠性。

5) 公共网络

用于提供数据库服务的网络环境。公共网络与内部网络应使用不同的交换机。

注意：DM7 读写分离系统中，主备数据库之间，数据库和守护进程之间，以及监视器与守护进程之间都是通过内部网络进行通信，用户应该确保内部网络是绝对可靠的，一旦内部网络不可靠或者拔、插主备库连接内部网络的网线，可能会导致监视器误判主库故障，此时如果执行接管主库命令，会导致系统中出现双主库情况，损坏数据库数据。

配置规划

规划配置一个主库实例带两个备库实例和一个监视器的读写分离主备系统。

表 32.8 配置环境说明

机器名	实例名	IP 地址	初始状态	操作系统	备注
TA_P	DM1	192.168.1.131 192.168.0.141	主库	Linux rh6-141.test 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.131 外部服务 IP； 192.168.0.141 内部通信 IP
TA_S1	DM2	192.168.1.132	即时备库	Linux	192.168.1.132 外

		192.168.0.142		rh6-142.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	部服务 IP；192.168.0.142 内部通信 IP
TA_S2	DM3	192.168.1.133 192.168.0.143	即时备库	Linux rh6-143.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.133 外部服务 IP；192.168.0.143 内部通信 IP
TA_M		192.168.0.144	监视器	Linux rh6-144.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.0.144 内部通信 IP

数据准备

配置数据守护之前，必须先同步主备库数据，确保两者保持完全一致；主数据库可以是新初始化的数据，也可以是正在生产、使用中的数据。DM7 提供了两种方式初始化同步主备库数据，数据文件拷贝以及备份还原方式。

不能使用分别初始化库的方法，原因如下：

1） 每个库都有一个永久魔数（permenant_magic），主备库传送日志时会判断这个值是否一样，确保来自同一个库，不同的库传送不了日志

2） 由于 dminit 初始化数据库时，会生成随机密钥用于加密，每次生成的密钥都不相同，备库无法解析采用主库密钥加密的数据

数据文件拷贝

如果搭建系统之前，数据库系统已经上线运行了，那么我们可以通过拷贝数据文件的方式实现主备数据库的同步。具体步骤包括：

1．正常关闭数据库

2．严格按照数据文件在主库上的分布，拷贝数据文件到备库的对应目录

3．如果数据文件统一存放在一个目录下，则直接拷贝整个目录即可

备份还原方式

用户也可以通过脱机备份、脱机还原的方式同步主备库数据，更详细的说明可以参考备份恢复相关文档。具体步骤包括：

1．正常关闭数据库

2．进行脱机备份

```
   ./dmbackup        TYPE=FULL        INI_PATH=/dm7data/DM1/DAMENG/dm.ini
NAME=BACKUP_FILE
```

3．拷贝备份文件到备库

4．执行脱机数据库恢复

```
   ./dmrestore                        INI_PATH=/dm7data/DM1/DAMENG/dm.ini
FILE=/dm7data/DM1/DB_DAMENG_20120910180136000741.bak
```

检查数据一致性

切换到执行码安装目录/dm7/bin，分别以 mount 方式启动数据库

```
./dmserver /dm7data/DM1/DAMENG/dm.ini mount
```

启动命令行工具 disql，分别连接登录主备库实例查询 LSN 值

```
SQL>select file_lsn, cur_lsn from v$rlog;


行号      FILE_LSN           CUR_LSN
--------- ------------------ --------------------
1         10250              10250
```

比较各个实例的 FILE_LSN 和 CUR_LSN 值，需要确保主备数据库的数据完全一致，如果不一致系统将会启动失败，需要重新准备数据。

分别查询主备库，确保永久魔数是否一致

```
SQL>select permanent_magic;

行号       PERMANENT_MAGIC
---------- ---------------
1          1385101984
```

正常关闭主备数据库，继续配置。

配置主数据库

配置主数据库的过程包括修改配置文件 dm.ini、dmmal.ini、dmarch.ini、dmrww.ini。所有配置文件都位于/dm7data/DM1/DAMENG 目录下。

配置 dm.ini

配置主数据库实例名为 DM1，dm.ini 参数修改如下：

```
INSTANCE_NAME           = DM1
PORT_NUM                = 5236

DW_UDP_PORT     = 5243
INST_UDP_PORT   = 5253

MAIL_INI                = 1
ARCH_INI                = 1
```

配置 dmmal.ini

配置 MAL 系统，主备数据库的 dmmal.ini 配置完全一致，MAL_HOST 使用内部网络 IP，MAL_INST_HOST 使用外部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，配置如下：

```
MAL_CHECK_INTERVAL      = 5
MAL_CONN_FAIL_INTERVAL  = 5
[MAL_INST1]
  MAL_INST_NAME         = DM1
  MAL_HOST              = 192.168.0.141
  MAL_PORT              = 5336
  MAL_INST_HOST         = 192.168.1.131
  MAL_INST_PORT         = 5236
[MAL_INST2]
  MAL_INST_NAME         = DM2
  MAL_HOST              = 192.168.0.142
  MAL_PORT              = 5337
  MAL_INST_HOST         = 192.168.1.132
  MAL_INST_PORT         = 5237
[MAL_INST3]
  MAL_INST_NAME         = DM3
  MAL_HOST              = 192.168.0.143
  MAL_PORT              = 5338
  MAL_INST_HOST         = 192.168.1.133
  MAL_INST_PORT         = 5238
```

配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档，即时归档的 ARCH_DEST 为备数据库实例名，这里除了配置本地归档外，需要配置到其他所有实例的即时归档，以保证备库接管

为新主库后可以正常向其它实例发送即时归档。

ARCH_WAIT_APPLY 参数表示是否需要等待备库做完日志，取值 0 和 1,默认 1，所有主备库的 dmarch.ini 中，该值都要保持一致。

```
ARCH_WAIT_APPLY      = 1    #默认 1，表示需要等待备库做完日志

[ARCHIVE_TIMELY1]
 ARCH_TYPE              = TIMELY
 ARCH_DEST              = DM2

[ARCHIVE_TIMELY2]
 ARCH_TYPE              = TIMELY
 ARCH_DEST              = DM3

[ARCHIVE_LOCAL1]
 ARCH_TYPE              = LOCAL
 ARCH_DEST              = /dm7data/DM1/DAMENG/arch
 ARCH_FILE_SIZE         = 128    #单位 Mb
 ARCH_SPACE_LIMIT       = 0      #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

一般本地归档 ARCH_SPACE_LIMIT 需要根据实际的磁盘空间大小进行设置。当磁盘空间不够时服务器挂住，直到用户干预，释放出磁盘空间后系统继续运行。

配置 dmrww.ini

和 dmwatch.ini 配置项相比，只需要配置实例自身相关信息即可：

1.UDP 广播配置方式

```
[GROUP_A]
 RWW_BRO_ADDR           = 192.168.0.255
 RWW_UDP_PORT           = 5243
 RWW_INST_ERROR_TIME    = 5
 RWW_FAR_ERROR_TIME     = 5
 RWW_MODE               = 1

 INST_OGUID             = 82379
 INST_NAME              = DM1
INST_UDP_PORT           = 5253
INST_STARTUP_CMD        = /dm7/bin/dmserver /dm7data/DM1/DAMENG/dm.ini
 INST_STARTUP_MODE      = 1        #0: 服务方式启动； 1：控制台方式启动
 INST_AUTO_RESTART      = 1        #实例故障后是否自动重启
```

2. UDP 组播配置方式

```
[GROUP_A]
 RWW_UDP_TYPE           = 1   #组播方式
 RWW_UDP_TTL            = 64   #组播包生存期
 RWW_BRO_ADDR           = 234.1.1.2 #组播地址
```

```
 RWW_UDP_PORT              = 5243
 RWW_INST_ERROR_TIME    = 5
 RWW_FAR_ERROR_TIME     = 5
 RWW_MODE                  = 1

 INST_OGUID                = 82379
 INST_NAME                 = DM1
INST_UDP_PORT              = 5253
INST_STARTUP_CMD           = /dm7/bin/dmserver /dm7data/DM1/DAMENG/dm.ini
 INST_STARTUP_MODE         = 1          #0: 服务方式启动；  1：控制台方式启动
 INST_AUTO_RESTART         = 1          #实例故障后是否自动重启
```

3．TCP 配置方式

```
[GROUP_A]
 RWW_UDP_PORT              = 5243
 RWW_INST_ERROR_TIME    = 5
 RWW_FAR_ERROR_TIME     = 5
 RWW_MODE                  = 1

 RWW_TCP_PORT             = 6243
 RWW_MAL_INI              = /dm7data/DM1/DAMENG/dmmal.ini
 RWW_ARCH_INI             = /dm7data/DM1/DAMENG/dmarch.ini
 RWW_ARCH_TYPE            = TIMELY

 INST_OGUID                = 82379
 INST_NAME                 = DM1
INST_UDP_PORT              = 5253
INST_STARTUP_CMD           = /dm7/bin/dmserver /dm7data/DM1/DAMENG/dm.ini
 INST_STARTUP_MODE         = 1          #0: 服务方式启动；  1：控制台方式启动
 INST_AUTO_RESTART         = 1          #实例故障后是否自动重启
```

启动主数据库

以 mount 方式启动主数据库

```
./dmserver /dm7data/DM1/DAMENG/dm.ini mount
```

注意：一定要以 mount 方式启动数据库，否则系统启动时会重构回滚表空间，生成 REDO 日志；并且，启动后应用可能连接到数据库进行操作，破坏主备数据库的一致性。

设置 oguid

启动命令行工具 disql，登录主数据库设置 OGUID 值为 82379

```
SQL>sp_set_oguid(82379);
```

注意：系统通过 OGUID 值确定一个读写分离组，由用户保证 OGUID 值的唯一性，并确保读写分离系统中，数据库、dmrww 进程和监视器配置相同的 OGUID 值。

修改数据库模式

启动命令行工具 disql，登录主数据库修改数据库模式为 PRIMARY

```
SQL>alter database primary;
```

配置备数据库

配置备数据库的过程和主库大致相同，这里只介绍配置 DM2 实例的过程。包括修改配置文件 dm.ini、dmmal.ini、dmarch.ini、dmrww.ini。所有配置文件都位于 /dm7data/DM2/DAMENG 目录下。DM3 的配置只需仿照 DM2 配置过程即可。

配置 dm.ini

配置备数据库实例名为 DM2，dm.ini 参数修改如下：

```
INSTANCE_NAME          = DM2
PORT_NUM               = 5237

DW_UDP_PORT            = 5243
INST_UDP_PORT          = 5253

MAIL_INI               = 1
ARCH_INI               = 1
```

配置 dmmal.ini

备库的 dmmal.ini 配置文件与主库的 dmmal.ini 完全一致，拷贝过来即可。

配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档，即时归档对象为 DM1 和 DM3，配置 DM3 的 dmarch.ini 时，即时归档对象为 DM1 和 DM2。

ARCH_WAIT_APPLY 参数值要和主库一致。

```
 ARCH_WAIT_APPLY     = 1   #默认 1，表示需要等待备库做完日志

[ARCHIVE_TIMELY1]
```

```
 ARCH_TYPE               = TIMELY
 ARCH_DEST               = DM1

[ARCHIVE_TIMELY2]
 ARCH_TYPE               = TIMELY
 ARCH_DEST               = DM3

[ARCHIVE_LOCAL1]
 ARCH_TYPE               = LOCAL
 ARCH_DEST               = /dm7data/DM2/DAMENG/arch
 ARCH_FILE_SIZE          = 128    #单位 Mb
 ARCH_SPACE_LIMIT        = 0      #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

一般本地归档 ARCH_SPACE_LIMIT 需要根据实际的磁盘空间大小进行设置。当磁盘空间不够时服务器挂住，直到用户干预，释放出磁盘空间后系统继续运行。

配置 dmrww.ini

和 dmwatch.ini 配置项相比，只需要配置实例自身相关信息即可：

1．UDP 广播配置方式

```
[GROUP_A]
 RWW_BRO_ADDR            = 192.168.0.255
 RWW_UDP_PORT            = 5243
 RWW_INST_ERROR_TIME     = 5
 RWW_FAR_ERROR_TIME      = 5
 RWW_MODE                = 1

 INST_OGUID              = 82379
 INST_NAME               = DM2
 INST_UDP_PORT           = 5253
 INST_STARTUP_CMD        = /dm7/bin/dmserver /dm7data/DM2/DAMENG/dm.ini
 INST_STARTUP_MODE       = 1         #0: 服务方式启动；  1：控制台方式启动
 INST_AUTO_RESTART       = 1         #实例故障后是否自动重启
```

2．UDP 组播配置方式

```
[GROUP_A]
 RWW_UDP_TYPE            = 1   #组播方式
 RWW_UDP_TTL             = 64   #组播包生存期
 RWW_BRO_ADDR            = 234.1.1.2 #组播地址
 RWW_UDP_PORT            = 5243
 RWW_INST_ERROR_TIME     = 5
 RWW_FAR_ERROR_TIME      = 5
 RWW_MODE                = 1

 INST_OGUID              = 82379
```

```
INST_NAME                = DM2
INST_UDP_PORT            = 5253
INST_STARTUP_CMD         = /dm7/bin/dmserver /dm7data/DM2/DAMENG/dm.ini
INST_STARTUP_MODE        = 1        #0: 服务方式启动；  1：控制台方式启动
INST_AUTO_RESTART        = 1        #实例故障后是否自动重启
```

3.TCP 配置方式

```
[GROUP_A]
 RWW_UDP_PORT            = 5243
 RWW_INST_ERROR_TIME     = 5
 RWW_FAR_ERROR_TIME      = 5
 RWW_MODE                = 1

RWW_TCP_PORT             = 6243
RWW_MAL_INI              = /dm7data/DM2/DAMENG/dmmal.ini
RWW_ARCH_INI             = /dm7data/DM2/DAMENG/dmarch.ini
RWW_ARCH_TYPE            = TIMELY

 INST_OGUID              = 82379
 INST_NAME               = DM2
 INST_UDP_PORT           = 5253
 INST_STARTUP_CMD        = /dm7/bin/dmserver /dm7data/DM2/DAMENG/dm.ini
 INST_STARTUP_MODE       = 1        #0: 服务方式启动；  1：控制台方式启动
 INST_AUTO_RESTART       = 1        #实例故障后是否自动重启
```

启动备数据库

以 mount 方式启动备数据库

```
./dmserver /dm7data/DM2/DAMENG/dm.ini mount
```

注意：一定要以 mount 方式启动数据库，否则系统启动时会重构回滚表空间，生成 REDO 日志；并且，启动后应用可能连接到数据库进行操作，破坏主备数据库的一致性。

设置 oguid

启动命令行工具 disql，登录主数据库设置 OGUID 值为 82379

```
SQL>sp_set_oguid(82379);
```

注意：系统通过 OGUID 值确定一个读写分离组，由用户保证 OGUID 值的唯一性，并确保读写分离系统中，数据库、dmrww 进程和监视器配置相同的 OGUID 值。

修改数据库模式

启动命令行工具 disql，登录备数据库修改数据库模式为 STANDBY

```
SQL>alter database standby;
```

配置监视器

编辑 dmrwm.ini 配置监视器，可以在内部网络除主备库之外的任意一台机器上配置监视器， dmrwm.ini 配置文件中的配置项的值要与 dmrww.ini 对应配置项保持一致。

允许用户在多台机器上配置并启动 dmrwm 来监控系统，dmrwm 内部可以处理多个命令同时执行的冲突，但需要保证接管命令和被接管主库的恢复命令在同一个监视器上执行。

如果用户在机器 A 部署的监视器上执行接管命令，接管主库 P1，则会在 A 本地的控制文件中记录接管信息，如果在机器 B 的监视器上恢复主库 P1，在 B 本地的控制文件中会找不到 P1 的接管信息而恢复失败。

也可以所有的监视器共用同一个控制文件，但在执行接管和恢复前需要先拷贝到本地。

其他命令的使用没有此限制。

另外在接管前一定要确认是主库故障而不是内网网卡故障，以免出现双主库。

1. UDP 广播配置方式

```
RWM_BRO_ADDR        = 192.168.0.255
RWM_UDP_PORT        = 5243
RWM_INST_OGUID      = 82379
RWM_GROUP_NAME      = GROUP_A
RWM_CTL_PATH         = /dm7data

RWM_LOG_PATH        = /dm7data/log
RWM_LOG_INTERVAL    = 60        #单位 s
RWM_LOG_FILE_SIZE   = 64        #单位 Mb
RWM_LOG_SPACE_LIMIT = 0         #单位 Mb，0 表示无空间限制
```

2. UDP 组播配置方式

```
RWM_UDP_TYPE        = 1
RWM_UDP_TTL         = 64
RWM_BRO_ADDR        = 234.1.1.1
RWM_UDP_PORT        = 5243
RWM_INST_OGUID      = 82379
RWM_GROUP_NAME      = GROUP_A
RWM_CTL_PATH         = /dm7data

RWM_LOG_PATH        = /dm7data/log
RWM_LOG_INTERVAL    = 60        #单位 s
RWM_LOG_FILE_SIZE   = 64        #单位 Mb
RWM_LOG_SPACE_LIMIT = 0         #单位 Mb，0 表示无空间限制
```

3. TCP 配置方式

```
RWM_INST_OGUID        = 82379
RWM_GROUP_NAME        = GROUP_A
RWM_CTL_PATH           = /dm7data

RWM_RWW_TCP_PORT   = 6243
RWM_RWW_IP             = 192.168.0.141
RWM_RWW_IP             = 192.168.0.142
RWM_RWW_IP             = 192.168.0.143

RWM_LOG_PATH           = /dm7data/log
RWM_LOG_INTERVAL    = 60       #单位 s
RWM_LOG_FILE_SIZE    = 64       #单位 Mb
RWM_LOG_SPACE_LIMIT = 0       #单位 Mb，0 表示无空间限制
```

查看配置

配置完成后，连接数据库查询各个配置项，再次确认配置的正确性。

1．启动命令行工具 disql，登录主数据库，查看主数据库相关信息：

（1）查看归档是否打开

```
SQL>select arch_mode from v$database;

行号        ARCH_MODE
---------- ---------
1          Y
```

（2）查看归档日志信息

```
SQL>select arch_name,arch_type,arch_dest from v$dm_arch_ini;

行号        ARCH_NAME         ARCH_TYPE  ARCH_DEST
---------- ----------------  ---------  ----------------
1          ARCHIVE_TIMELY1   TIMELY     DM2
2          ARCHIVE_TIMELY2   TIMELY     DM3
3          ARCHIVE_LOCAL1    LOCAL      /dm7data/DM1/DAMENG/arch
```

（3）查看 MAL 系统信息

```
SQL>select * from v$dm_mal_ini;

行号        MAL_TYPE INST_NAME IP             MAL_PORT
---------- -------- --------- ------------- -----------
1          MAIL_INST1     DM1       192.168.0.141 5336
2          MAIL_INST2     DM2       192.168.0.142 5337
3          MAIL_INST3     DM3       192.168.0.143 5338
```

2．启动命令行工具 disql，登录备数据库 DM2，查看备数据库相关信息：

（1）查看归档是否打开

```
SQL>select arch_mode from v$database;

行号        ARCH_MODE
---------- ---------
1          Y
```

（2）查看归档日志信息

```
SQL>select arch_name,arch_type,arch_dest from v$dm_arch_ini;

行号        ARCH_NAME         ARCH_TYPE ARCH_DEST
---------- ---------------- --------- ----------------
1          ARCHIVE_TIMELY1   TIMELY    DM1
2          ARCHIVE_TIMELY2   TIMELY    DM3
3          ARCHIVE_LOCAL1    LOCAL     /dm7data/DM2/DAMENG/arch
```

（3）查看 MAL 系统信息

```
SQL>select * from v$dm_mal_ini;

行号        MAL_TYPE INST_NAME IP            MAL_PORT
---------- -------- --------- ------------- -----------
1          MAIL_INST1     DM1       192.168.0.141 5336
2          MAIL_INST2     DM2       192.168.0.142 5337
3          MAIL_INST3     DM3       192.168.0.143 5338
```

3．启动命令行工具 disql，登录备数据库 DM3，查看备数据库相关信息：

（1）查看归档是否打开

```
SQL>select arch_mode from v$database;
行号        ARCH_MODE
---------- ---------
1          Y
```

（2）查看归档日志信息

```
SQL>select arch_name,arch_type,arch_dest from v$dm_arch_ini;

行号        ARCH_NAME         ARCH_TYPE ARCH_DEST
---------- ---------------- --------- ----------------
1          ARCHIVE_TIMELY1   TIMELY    DM1
2          ARCHIVE_TIMELY2   TIMELY    DM2
3          ARCHIVE_LOCAL1    LOCAL     /dm7data/DM3/DAMENG/arch
```

（3）查看 MAL 系统信息

```
SQL>select * from v$dm_mal_ini;

行号        MAL_TYPE INST_NAME IP            MAL_PORT
---------- -------- --------- ------------ -----------
1          MAIL_INST1     DM1       192.168.0.141 5336
2          MAIL_INST2     DM2       192.168.0.141 5337
3          MAIL_INST3     DM3       192.168.0.141 5338
```

启动守护进程

启动主库 DM1 的守护进程。

```
./dmrww /dm7data/DM1/DAMENG/dmrww.ini
```

启动备库 DM2 的守护进程。

```
./dmrww /dm7data/DM2/DAMENG/dmrww.ini
```

启动备库 DM3 的守护进程。

```
./dmrww /dm7data/DM3/DAMENG/dmrww.ini
```

守护进程启动后，进入 STARTUP 状态，备库的守护进程直接 OPEN 本机实例，守护进程切换为 OPEN 状态，主库的守护进程接收所有备库的广播消息，判断所有备库都已 OPEN 后，OPEN 本地主库实例，并修改守护进程为 OPEN 状态。

启动监视器

启动监视器：

```
./dmrwm /dm7data/dmrwm.ini
```

至此 DM7 读写分离系统搭建完毕，可以提供完善的数据保护和高可用性特性。

使用监视器

提供 dmrwm 工具对读写分离主备系统进行管理，其功能包括启动系统、显示系统信息以及恢复系统故障等。

表 1 dmrwm 命令

命令	含义
HELP	显示帮助信息
SHOW	显示所有实例信息
SHOW I[NTERVAL] n	每隔n秒自动显示组中所有实例信息
Q	取消自动显示
LOGIN	登录 DMRWM
LOGOUT	退出登录
TIP	显示系统当前运行状况
STARTUP ALL	启动所有 dmrww 的监控功能

STOP ALL	停止所有 dmrww 的监控功能
STARTUP 实例名	启动指定实例dmrww的监控功能
STOP 实例名	停止指定实例dmrww的监控功能
INST STARTUP ALL	启动所有实例
INST EXIT ALL	退出所有实例
INST STARTUP 实例名	启动指定的实例
INST EXIT 实例名	退出指定的实例
OPEN ALL	open 所有实例
OPEN 实例名	open 指定的实例
SWITCHOVER 实例名	切换指定的实例为主库
RECOVER 实例名	恢复指定的实例
CHOOSE	选择可以接管主库的备库
TAKEOVER USE 实例名	使用指定的实例接管故障主库
TAKEOVER FORCE USE 实例名	使用指定的实例强制接管故障主库
EXIT	退出工具

命令参数:

```
dmrwm  [ini_path | help]
```

示例：

```
./dmrwm /dm7data/dmrwm.ini
```

注意：需要指定正确的 dmrwm.ini 路径，工具才能正常启动。

查看实例状态

SHOW：
显示系统中所有实例信息

SHOW I[NTERVAL]：
每隔 n 秒自动显示组中所有实例信息，n 取值 5~3600，单位为秒。
控制台有命令输入时会取新自动显示。

Q：
取消自动显示

登录 DMRWM

LOGIN:

监视器执行以下命令时都需要先登录，用户名/密码和服务器登录口令一致，且必须有 DBA 权限。

1. STOP ALL/STOP 实例名
2. INST EXIT ALL/INST EXIT 实例名

3．OPEN ALL/OPEN 实例名

4．SWITCHOVER/RECOVER/TAKEOVER USE 实例名

其他命令不需要登录监视器。

LOGIN 命令默认使用 SYSDBA/SYSDBA 作为登录口令，可以直接回车，不需要显示输入。

退出登录

LOGOUT:

退出登录监视器。

停止/启动所有监控

1. STOP ALL

（1） 停止所有 dmrww 的监控功能

（2） 如果某个 dmrww 已经是 SHUTDOWN 状态，会跳过继续执行

2. STARTUP ALL

（1） 启动所有 dmrww 的监控功能

（2） 如果某个 dmrww 已经是 STARTUP 或 OPEN 状态，会跳过继续执行

停止/启动指定实例的监控

1. STOP 实例名

停止指定实例 dmrww 的监控功能

2. STARTUP 实例名

启动指定实例 dmrww 的监控功能

启动所有实例

INST STARTUP ALL:

启动所有实例，如果 dmrww 已经配置有自动重启，则命令会执行失败，等待 dmrww 自动将实例拉起。

如果某个实例已经处于运行状态，则跳过继续启动其他实例，主备库启动没有先后顺序，主库等所有备库 OPEN 后自动 OPEN。

启动指定的实例

INST STARTUP 实例名：

启动指定的实例，如果 dmrww 已经配置有自动重启，则命令会执行失败，等待 dmrww 自动将实例拉起。

要求实例处于 SHUTDOWN 状态，dmrww 为 STARTUP 或 SHUTDOWN 状态。

退出所有实例

INST EXIT ALL:

退出系统中的所有实例，为防止退出后 dmrww 又将实例自动拉起， dmrww 会切换为 SHUTDOWN 状态，dmrww 不会退出，如果命令执行失败，会将 dmrww 重新设置为 STARTUP 状态。

该命令用于系统正常运行情况下的正常退出，执行命令前所有守护进程和实例都必须处于 OPEN 状态。

退出指定的实例

INST EXIT 实例名：

退出指定的实例，为防止退出后 dmrww 又将实例自动拉起， dmrww 会切换为 SHUTDOWN 状态，dmrww 不会退出，如果命令执行失败，会将 dmrww 重新设置为 STARTUP 状态。

该命令执行时，要求实例正常运行，实例 dmrww 状态为 STARTUP 或 OPEN，否则不允许退出实例。

OPEN 实例

1. OPEN ALL

（1） open 所有实例，执行顺序为先 open 所有备库，再 open 主库

（2） 系统中如果已存在 open 状态的实例，open all 会跳过继续 open 其他实例

2. OPEN + 实例名

（1） open 指定的实例，主库故障重启，仍然作为主库重加入时，可以直接 open。

（2） 备库故障重启，也可以直接 open，如果 lsn 小于主库，即时归档状态会转为无效状态，需要使用 RECOVER 命令执行恢复。

注意：这两个命令一般不需要使用。

（1） 对于备库，mount 方式启动后，dmrww 会自动执行 open 操作。

（2） 对于主库，mount 方式启动后，dmrww 根据接收到的备库广播消息来判断是否自动 open。

a. 主库需要收到所有备库 dmrww 的广播消息，如果备库的 dmrww 故障，主库不会

自动 open。

b. 如果有备库不是 standby&open 状态，主库不会自动 open

如果确实是备库出现故障，且用户可以确认系统中只有一个主库，且主库数据可靠的情况下，可以使用 open 命令单独 open 主库。

但在情况 a 下，主库 dmrww 会一直保持 STARTUP 状态，不允许执行自动或手动恢复操作。

系统中存在多个主库实例时：

（1） 多个 mount 状态的 primary 实例

a. 不允许执行 open all

b. 允许执行 open 操作，只允许 open pocnt 计数最大的主库实例（最后一个执行接管操作的主库），其他实例 open 会报错“已被接管”。

（2） 多个 open 状态的 primary 实例

会强制 shutdown 所有 open 状态的主库

pocnt 计数：非 standby 模式的实例每执行一次 open 操作时该计数加 1，在没有用户干预的情况下，认为系统中 open 次数最多的主库实例为合法主库。

恢复实例

RECOVER + 实例名：

（1） 恢复故障主库

查找控制文件，判断主库 lsn 是否小于接管当时备库的 lsn，若符合则作为备库重新加入，否则报错处理。

（2） 恢复故障备库

通知主库 dmrww 修改为 recovery 状态，并发送即时归档到备库，备库和主库数据一致后 open 备库。

注意：dmrww 在自动守护模式（RWW_MODE=1）下，会自动检测并恢复故障备库，故障主库仍然需要使用 recover 命令手动恢复。

选择接管备库

CHOOSE：

监视器选择出可以接管故障主库的即时备库。

注意：这里只选择即时归档状态为 VALID 的备库，如果系统中所有备库即时归档都无效，则不允许接管主库，如果要强制接管，则需要使用强制接管命令。

接管命令

TAKEOVER USE + 实例名：

主库故障后，使用指定的备库实例接管为新主库，并记录接管信息到控制文件中。

此命令要求接管用的备库即时归档状态是 VALID 有效状态，接管完成后，可能会出现故障主库无法重新加入的情况。
如果主库是在写入本地日志之后，发送即时归档到备库之前故障，备库接管时会比主库少一段日志，在接管完成后，不允许故障主库再加入系统，避免数据被破坏。

注意：使用接管命令时，一定要确认是主库实例故障，而非内网网络问题或网卡故障导致接收不到实例广播消息而出现的误判，否则直接接管可能会造成系统中出现双主库，导致数据被破坏。

强制接管命令

TAKEOVER FORCE USE + 实例名：
主库故障后，允许使用此命令强制接管故障主库，此命令不要求接管用的备库即时归档状态为 VALID，但必须是所有备库（监视器收到过消息的）中 SLSN 值最大的备库，且处于 OPEN 状态。

使用此命令需要谨慎，可能会导致以下情况：

如果执行接管的备库即时归档状态是 INVALID，则和故障主库之间有日志差，强制接管后，故障主库将无法再重新加入主备系统，需要重新准备数据。

如果接管当时，有 LSN 更大的故障备库（监视器没有收到过此故障备库的消息），在接管成功后，该故障备库又重新加入系统，接管后的新主库做了一系列操作后，LSN 比重加入的故障备库大，又再次同步数据过去，则会导致数据被破坏。

注意：使用接管命令时，一定要确认是主库实例故障，而非内网网络问题或网卡故障导致接收不到实例广播消息而出现的误判，否则直接接管可能会造成系统中出现双主库，导致数据被破坏。

切换主备库

SWITCHOVER + 实例名：
主备库正常的情况下，将指定的备库实例切换为新主库，老的主库切换为备库。

注意：执行切换时要求主备库实例都是 open 状态，且主备库 dmrww 也处于 open 状态，否则不允许切换。

如果切换过程中，待切换为主库的备库故障，而原主库已切换为备库模式，switchover 执行失败，导致系统中没有可用主库时，可以通过接管方法，选择一个符合条件的备库接管为新主库。

状态检测

TIP：检测系统当前运行状况。

主要做下面三个方面检查：

（1） 系统中是否存在多个 open 状态的主库

（2） 系统中主库状态

a. 主库实例是否正常（primary&open）

b. 主库 lsn 是否大于等于备库

c. 主库 dmrww 状态是否正常

（3） 系统中各备库状态

a. 备库实例是否正常（standby&open）

b. 备库即时归档是否有效

c. 备库 dmrww 状态是否正常

如果检查到存在异常，在 rwm 端打印相关提示信息给用户。

帮助信息

HELP：

显示 dmrwm 各种命令的使用格式

退出监视器

EXIT：

退出监视器

故障处理

主库故障

1. **主库服务意外停掉，可以 mount 重启成功**
   a. 备库都是 standby&open 情况下，主库重启后，dmrww 会自动 open。
   b. 系统中存在故障备库，DBA 确保主库数据可靠的前提下，可以通过 dmrwm 工具“open+实例名”的方式将主库 open。

2. **主库故障，无法重启：**
   a. 使用 dmrwm 工具“choose”命令选择出可以接管主库的即时备库
   b. 使用 dmrwm 工具“takeover use + 实例名”命令，使用即时备库接管故障主库

注意这里 choose 命令选出的备库要求即时归档都是有效状态，如果 choose 选出结果为空，用户想强制接管的话，可以通过 disql 方式登录备库，mount 备库，修改为主库模式并重新 open，但这样可能会发生数据丢失，需要用户来保证。

3. 主库作为备库重加入：

使用 dmrwm 工具“recover + 实例名”的方式，根据监视器控制文件中记录的 lsn 值判断主库是否能重新加入，如果可以，则做为即时备库加入，否则不允许恢复，故障主库不再可用。

备库故障

1. 自动守护模式

该模式下主库 dmrww 会检测各备库状态，如果满足恢复条件则会自动执行恢复操作，有多个待恢复备库时，优先恢复 lsn 最大的备库，保证尽快提供出可用备库。

2. 非自动守护模式

该模式下，需要通过 dmrwm 的 recover 命令，手动恢复指定的备库实例。

多主库处理

1. 多个 open 状态的主库

dmrww 会发送 shutdown 命令，强制关闭所有 open 主库。

dmrwm 端不允许执行 open / open all / choose / takeover / switchover / recover 操作

2. 一个 open，多个非 open 状态的主库

检查 open 状态主库的 pocnt 值，如果小于某个非 open 状态的主库，则执行 shutdown 操作，强制关闭 open 状态的主库。

否则认为当前 open 主库是合法的，dmrwm 端不允许 open 其他主库，不允许执行 open all，其他命令可以正常执行。

注意：由于广播消息及主库的关闭过程中都存在延时，主库被强制 shutdown 后无法保证其数据的可靠性，且多个 open 主库的情况下，会同时向即时备库发送归档，可能备库数据已经被破坏，需要考虑是否重新搭建环境。

客户端连接

读写分离系统对于应用程序来讲是完全透明的。应用程序仅需要通过客户端配置读写分离属性，连接到主库数据库服务器，就可以自动实现读写分离了。

接口实现中，对每个客户端，当连接到主库后，主库会返回一个正常的备库地址和端口号，再利用此信息连接到备库，因此实际上每个连接都是一主一备的模式。如果备库出现故障，连接断开，相应主库上的连接也都会被断开，需要客户端重新再建立连接，由主库返回新的可用备库给客户端，如果所有备库都不可用，则直接在主库上执行读写操作。

因此，建议应用程序加上自动重连处理，防止当前连接的备库故障时对外服务中断。

客户端自动重连设置通过编辑 dm_svc.conf 文件，配置连接服务名。dm_svc.conf 配置文件在 DM7 安装时生成，Windows 平台下位于%SystemRoot%\system32 目录，Linux 平台下位于/etc 目录。

连接服务名格式：

```
SERVERNAME=(IP[:PORT],IP[:PORT],.....)
```

配置项目说明：

- SERVERNAME　　连接服务名，用户通过连接服务名访问数据库
- IP　　数据库所在的 IP 地址，如果是 IPv6 地址，为了区分端口，需要用[]封闭 IP 地址
- PORT　　数据库使用的 TCP 连接端口，可选配置，不配置则使用连接上指定的端口

另外下面几个参数：

```
LOGIN_PRIMARY=(1/0)
SWITCH_TIME=(val)
SWITCH_INTERVAL=(val)
```

用来配置在服务器之间切换连接的，它们可配合 SERVERNAME 一起使用。

1) LOGIN_PRIMARY 表示是否仅登录到主库，支持的选项为 1（只连接主库）和 0（主库不存在的情况下可连接备库）。可以不指定，若不指定，默认值为 1；

2) SWITCH_TIME 表示在服务器之间切换的次数，有效值范围 1~9223372036854775807，默认值为 3；

3) SWITCH_INTERVAL 表示在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~9223372036854775807，默认值为 200；

这里我们配置一个名为 dw_svc 的连接服务名，使用 dw_svc 连接数据守护中的数据库，即可实现故障自动重连。

```
dw_svc=(192.168.1.131:5236,192.168.1.132:5236)
```

关于 dm_svc.conf 的详细设置，请参考《DM7 系统管理员手册》2.1.1.4 dm_svc.conf 章节。

以下说明客户端连接服务器时如何设置读写分离属性，详细请参考《DM7 程序员手册》对应的接口说明。

1. JDBC 接口

JDBC 客户端的自动读写分离，在连接串中增加了两个连接属性：

rwSeparate：取值 0 和 1，默认 0，表示不使用读写分离方式

rwPercent： 取值 0~100，默认值 25。表示分发总事务的百分之多少到主库上执行。

<DRIVER>dm.jdbc.driver.DmDriver</DRIVER>

<URL>jdbc:dm://192.168.0.206:5236?rwSeparate=1&rwPercent=1</URL>

2. DPI 接口

DPI 连接句柄上的属性：

DSQL_ATTR_RWSEPARATE	读写分离（可读写）
DSQL_ATTR_RWSEPARATE_PERCENT	读写分离比例（可读写）

3. DCI 接口

DCI 连接句柄上的属性：

OCI_ATTR_RW_SEPARATE	是否配置读写分离： TRUE，FALSE
OCI_ATTR_RW_SEPARATE_PERCENT	读写分离的比例：0~100

4. ODBC 接口

ODBC 的连接关键字：

RW_SEPARATE	是否配置读写分离：TRUE，FALSE
RW_SEPARATE_PERCENT	读写分离的比例：0~100

5. Provider 接口

DM .NET Provider 接口主要实现了 DmConnection

DmConnection 对象表示一个 DM 数据库打开的连接

RwSeparate：是否读写分离，有效值为 true 或者 false；

RwPercent：表示分发到主库的事务占主备库总事务的百分比，有效值范围：0~100，默认值为 25。

附录

附录 1 系统视图

用于主备环境下的一些系统信息查询：

1. V$UTSK_SYS

查询守护进程向服务器发送请求的执行情况。

注意在 RUNNING 字段值为 TRUE 时，DSEQ、CODE 和 SEND_LSN 的值才有意义，CAN_TAKEOVER 和 AUTO_SWITCH 字段只用于普通主备系统，读写分离主备和 MPP 主备下这两个字段没有意义，都是 FALSE。

序号	列	数据类型	说明
1	RUNNING	VARCHAR(64)	当前系统是否正在处理守护进程请求，TRUE 表示正在处理，FALSE 表示没有。
2	DSEQ	INTEGER	发送报文序号
3	CODE	INTEGER	DSEQ 对应的执行结果
4	SEND_LSN	BIGINT	主库发送给备库的最大归档 LSN
5	CAN_TAKEOVER	VARCHAR(64)	非自动切换模式下，能否 takeover 主库，TRUE 允许执行 takeover，FALSE 不允许执行
6	AUTO_SWITCH	VARCHAR(64)	Watch 的切换模式，TRUE 自动切换，FALSE 非自动切换

2. V$RECOVER_STATUS

该视图需要在主库上查询，用于查询备库的恢复进度，如果已恢复完成，查询结果为空。

注意这里显示的是主库向备库发送日志的进度，由于备库重做日志也需要时间，在最后一批日志发送完成后，KBYTES_TO_RECOVER 为 0，RECOVER_PERCENT 为 100%，表示主库已经完成所有日志发送，需要等待备库将最后一批日志重做完成。

RECOVER_PERCENT = (KBYTES_TOTAL - KBYTES_TO_RECOVER)/ KBYTES_TOTAL

序号	列	数据类型	说明
1	PRIMARY_NAME	VARCHAR(256)	主库实例名
2	STANDBY_NAME	VARCHAR(256)	主库正在恢复的备库实例名
3	PRIM_LSN	BIGINT	主库当前的 FLSN
4	SEND_LSN	BIGINT	主库已发给备库的最大 LSN
5	KBYTES_TOTAL	BIGINT	主库本地的日志总量，单位为 KB
6	KBYTES_TO_RECOVER	BIGINT	主库需要发送给备库的日志量，单位为 KB
7	FILES_TO_RECOVER	INTEGER	主库需要发给备库重做的日志文件个数
8	RECOVER_PERCENT	VARCHAR(64)	主库向备库已发送日志的比例

3. V$RAPPLY_SYS

该视图需要在备库上查询，用于查询备库重做日志时的一些系统信息。

序号	列	数据类型	说明
1	APPLYING	VARCHAR(64)	备库的重做线程是否正在处理任务，查询结果为 TRUE 或 FALSE
2	SEC_MAX_LSN	BIGINT	备库收到的第二大 LSN 值
3	MAX_LSN	BIGINT	备库收到的最大 LSN 值
4	TASK_NUM	INTEGER	备库上的重做任务数
5	PRIMARY_NAME	VARCHAR(128)	主库实例名
6	HAS_KEEP_BUF	VARCHAR(64)	备库上是否有 KEEP_BUF，查询结果为 TRUE 或 FALSE

4. V$RAPPLY_LOG_TASK

该视图需要在备库上查询，用于查询备库所有重做任务的日志信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	日志类型
2	MIN_LSN	BIGINT	日志的最小 LSN 值
3	MAX_LSN	BIGINT	日志的最大 LSN 值
4	PTX_OFF	SMALLINT	重做日志的 PTX 偏移
5	LOG_SIZE	INTEGER	日志长度，单位为字节

5. V$ARCH_FILE

查询本地归档日志信息。

序号	列	数据类型	说明
1	DB_MAGIC	INTEGER	数据库的 MAGIC 值
2	STATUS	VARCHAR(64)	归档日志状态

3	LEN	BIGINT	日志长度，单位为字节
4	FREE	BIGINT	写日志偏移位置
5	ARCH_LSN	BIGINT	归档文件起始 LSN，仅对归档文件有意义
6	CLSN	BIGINT	检查点的 LSN
7	ARCH_SEQ	INTEGER	归档文件起始 SEQ，仅对归档文件有意义
8	NEXT_SEQ	INTEGER	下一个 SEQ 值
9	CREAT_TIME	TIMESTAMP	归档文件创建时间
10	CLOSE_TIME	TIMESTAMP	归档文件关闭时间
11	PATH	VARCHAR(256)	归档文件所在路径，包括归档文件名

6. V$ARCH_STATUS

查询归档状态信息，包括本地归档和远程归档。

序号	列	数据类型	说明
1	ARCH_TYPE	VARCHAR(256)	归档类型
2	ARCH_DEST	VARCHAR(256)	归档目标，本地归档为归档路径，其他类型为远程实例名。
3	ARCH_STATUS	VARCHAR(256)	归档状态，VALID 为有效状态，INVALID 为无效状态

7. V$MAL_LINK_STATUS

查询本地实例到远程的 MAL 连接状态。

序号	列	数据类型	说明
1	SRC_SITE	VARCHAR(256)	源站点实例名
2	DEST_SITE	VARCHAR(256)	目的站点实例名。
3	CTL_LINK_STATUS	VARCHAR(256)	控制链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。
4	DATA_LINK_STATUS	VARCHAR(256)	数据链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。

附录 2 监视器接口

监视器提供动态链接库dmrwm_dll.dll/libdmrwm.so，供用户编程使用。

在调用接口时需要注意，由于监视器是根据守护进程发送过来的UDP或TCP消息来判断是否符合执行条件，如果监视器尚未收到消息，或者收到的UDP消息不可靠，很有可能会导致接口执行失败，可能需要尝试多次调用。

1.1 返回值说明

监视器接口总体的返回值策略为：小于 0 表示执行失败，其他值表示执行成功。对于启动/关闭守护进程监控和实例的相关接口，返回值为 100 表示执行前状态已经和预期结果一致，接口实际并未执行。相关的错误码说明请参考 1.8 小节。

1.2 C 接口说明

由于监视器可以检测守护进程或实例的状态变化，同时也允许执行命令来管理守护系统，为了能够在接口调用时获取到系统的状态变化信息，或者执行命令时产生的步骤信息，监视器提供有等待和获取信息相关接口，具体用法见以下说明。

1. 分配操作句柄

函数原型：

```
DMRWM_RETURN
dmrwm_alloc_handle(
    mhandle*        mhdl
);
```

功能说明：

分配监视器操作句柄，监视器的接口都通过此句柄调用执行。

参数说明：

mhdl: 输出参数，分配到的句柄

返回值：

0：分配成功

<0：分配失败

2. 初始化监视器环境

函数原型：

```
DMRWM_RETURN
dmrwm_init(
    mhandle         mhdl,
    mchar*          ini_path,
    mchar*          log_path
);
```

功能说明：

初始化监视器的环境

参数说明：

mhdl: 输入参数，监视器操作句柄

ini_path: 输入参数，指定 dmrwm.ini 的绝对路径

log_path: 输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则不生成日志文件

返回值：

0：执行成功

<0：执行失败

3. 等待消息事件

函数原型：

```
void
dmrwm_msg_event_wait(
    mhandle             mhdl
);
```

功能说明：

监视器检测到守护进程或实例状态发生变化，或者接口命令执行过程中，产生中间输出消息时会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等待事件发生后，可通过接口 dmrwm_get_exec_msg 去读取中间结果。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

无

4. 获取输出信息

函数原型：

```
DMRWM_RETURN
dmrwm_get_exec_msg(
    mhandle             mhdl,
    mchar*              buf_msg,
    muint4              buf_len,
    muint4*             msg_len_out,
    muint4*             get_flag
);
```

功能说明：

获取输出信息，和 dmrwm_msg_event_wait 配合使用。

参数说明：

mhdl:

输入参数，监视器操作句柄

buf_msg：

输出参数，保存获取到的输出消息，注意 buf_msg 缓存长度需要大于 2048，以免长度不够导致消息被截断。

buf_len:

输入参数，指定 buf_msg 可写入的最大长度，建议 buf_msg 缓存长度及 buf_len 输入值大于 2048，避免消息被截断。

msg_len_out:

输出参数，写消息成功后，实际写入的消息长度保存到 msg_len_out 中。

get_flag:

输出参数，是否继续读取消息，TRUE 表示还有消息可读，FALSE 表示已全部读完。

返回值：

0：执行成功

<0：执行失败

5. 设置自动显示间隔

函数原型：

```
DMRWM_RETURN
dmrwm_set_show_interval(
    mhandle             mhdle,
    mint4               show_interval
);
```

功能说明：

监视器根据指定的 show_interval 值，每隔 show_interval 个时间间隔，自动输出各实例的状态信息给用户，需要使用 dmrwm_get_exec_msg 接口获取到输出消息，show_interval 单位为秒，默认值为 0。

参数说明：

mhdle: 输入参数，监视器操作句柄

show_interval: 输入参数，设置定时输出状态信息的时间间隔，单位为秒，取值为 0，或 5~3600，取值为 0 表示取消自动输出，默认值为 0。

返回值：

<0：执行失败

其他值：执行成功

6. 获取自动显示间隔

函数原型：

```
DMRWM_RETURN
dmrwm_get_show_interval(
    mhandle             mhdle
);
```

功能说明：

获取当前设置的显示时间间隔，单位为秒。

参数说明：

mhdle: 输入参数，监视器操作句柄

返回值：

<0：获取失败

其他值：当前设置的自动显示时间间隔

7. 设置自动写入日志文件间隔

函数原型：

```
DMRWM_RETURN
dmrwm_set_log_interval(
    mhandle         mhdle,
    mint4           log_interval
);
```

功能说明：

监视器按照设置的 log_interval 值，每隔 log_interval 个时间间隔，自动将各实例的状态信息写入到日志文件，log_interval 单位为秒，默认值为 1。

参数说明：

mhdle: 输入参数，监视器操作句柄

log_interval: 输入参数，设置定时写入状态信息到日志文件的时间间隔，单位为秒，取值为 0，1 或 5~3600。

取值为 0 表示不写入任何信息到日志文件。

取值为 1 表示只写入普通的状态变化日志，例如收到新的信息或者检测到状态变化等。

取值为 5~3600 范围内的数值时，除了生成普通日志信息外，会按照设置的间隔定期写入站点状态信息到日志文件中。

返回值：

<0：执行失败

其他值：执行成功

8. 获取自动写入日志文件间隔

函数原型：

```
DMRWM_RETURN
dmrwm_get_log_interval(
    mhandle         mhdle
);
```

功能说明：

获取当前设置的自动写入日志的时间间隔，单位为秒。

参数说明：

mhdle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

9. 设置单个日志文件大小

函数原型：

```
DMRWM_RETURN
dmrwm_set_log_file_size(
    mhandle         mhdle,
    muint4          log_file_size
);
```

功能说明：

调用此接口，可以设置监视器单个的日志文件大小，达到最大值后，会自动生成并切换到新的日志文件中，如果达到最大的日志总空间限制，会自动删除创建时间最早的日志文件，日志文件命名方式为“dmrwm_端口号_年月日时分秒.log”，例如“dmrwm_55681_20150418230523.log”。

参数说明：

mhdle: 输入参数，监视器操作句柄

log_file_size: 输入参数，设置单个日志文件大小，单位为 M，取值范围为 16~2048，默认值为 64。

返回值：

<0：执行失败

其他值：执行成功

10. 获取单个日志文件大小

函数原型：

```
DMRWM_RETURN
dmrwm_get_log_file_size(
    mhandle             mhdle
);
```

功能说明：

获取当前设置的单个日志文件大小，单位为 M。

参数说明：

mhdle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

11. 设置日志总空间大小

函数原型：

```
DMRWM_RETURN
dmrwm_set_log_space_limit(
    mhandle             mhdle,
    muint4              log_space_limit
);
```

功能说明：

调用此接口，可以设置监视器的日志总空间大小，达到设置的总空间大小后，会自动删除创建时间最早的日志文件。

参数说明：

mhdle: 输入参数，监视器操作句柄

log_space_limit: 输入参数，设置日志总空间大小，单位为 M，取值为 0 或 256~4096，默认值为 0，表示没有空间限制。

返回值：

<0：执行失败

其他值：执行成功

12. 获取日志总空间大小

函数原型：

```
DMRWM_RETURN
dmrwm_get_log_space_limit(
    mhandle             mhdle
);
```

功能说明：

获取当前设置的日志总空间大小，单位为 M。

参数说明：

mhdle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

13. 登录监视器

函数原型：

```
DMRWM_RETURN
dmrwm_login(
    mhandle             mhdl,
    mchar*              username,
    mchar*              password
);
```

功能说明：

登录监视器，这里的登录口令和服务器的登录口令是一致的，且必须有 DBA 权限。

校验登录信息时由守护进程转发给服务器校验，并返回校验结果给监视器，因此需要保证系统中至少有一个活动实例，否则无法校验成功，

参数说明：

mhdl: 输入参数，监视器操作句柄

username: 输入参数，登录用户名

password: 输入参数，登录密码

返回值：

0：执行成功

<0：执行失败

14. 获取监控组信息

函数原型：

```
DMRWM_RETURN
dmrwm_get_group_info(
    mhandle          mhdl,
    mchar*           group_name,
    muint4*          oguid,
    mchar*           comm_type,
    muint4*          mon_port,
    mchar*           primary_name,
    muint2*          n_site
);
```

功能说明：

获取监控组信息

参数说明：

mhdl: 输入参数，监视器操作句柄

group_name: 输出参数，监控组名

oguid: 输出参数，组的 oguid 值

comm_type: 输出参数，监视器的通信方式

mon_port: 输出参数，监视器的通信端口

primary_name: 输出参数，组中的主库实例名

n_site: 输出参数，组中的实例个数

注意：该接口中所有的字符串输出参数，长度必须大于 16，以免发生溢出。

返回值：

0：执行成功

<0：执行失败

15. 获取系统当前状态

函数原型：

```
DMRWM_RETURN
dmrwm_tip(
    mhandle          mhdl
);
```

功能说明：

调用此接口返回当前系统的状态信息，通过 dmrwm_get_exec_msg 接口可以获取返回消息，如果有日志文件也可以通过读日志文件获取。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

16. 获取实例个数

函数原型：

```
DMRWM_RETURN
dmrwm_get_n_inst(
    mhandle          mhdl,
    muint2*          n_inst
);
```

功能说明：

获取监控组中的实例个数。

参数说明：

mhdl: 输入参数，监视器操作句柄

n_inst: 输出参数，返回实例个数

返回值：

0：执行成功

<0：执行失败

17. 获取指定序号的实例名

函数原型：

```
DMRWM_RETURN
dmrwm_get_nth_inst_name(
    mhandle          mhdl,
    mint2            nth,
    mchar*           inst_name
);
```

功能说明：

获取第 nth 个实例的实例名。由 dmrwm_get_n_inst 和 dmrwm_get_group_info 都可以获取到组中的实例个数 n_inst，nth 取值范围为 0 到 n_inst-1。

参数说明：

mhdl: 输入参数，监视器操作句柄

nth: 输入参数，取第 nth 个实例的实例名

inst_name: 输出参数，返回实例名称

返回值：

0：执行成功

<0：执行失败

18. 根据实例名获取站点信息

函数原型：

```
DMRWM_RETURN
dmrwm_get_inst_info_by_name(
    mhandle          mhdl,
    mchar*           inst_name,
    mchar*           group_name,
    muint4*          oguid,
    mchar*           rww_status,
    mchar*           inst_ok,
    mchar*           istatus,
    mchar*           imode,
    mchar*           trstat,
    mchar*           mcstat,
    mchar*           pname,
    muint4*          pocnt,
    muint8*          flsn,
    muint8*          clsn,
    muint8*          sslsn,
    muint8*          slsn,
    muint4*          dseq,
    mint4*           code,
    muint4*          mon_dseq,
    mint4*           mon_code
);
```

功能说明：

根据指定的实例名，获取对应的站点信息

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定要获取的实例名称

group_name: 输出参数，监控组名

oguid: 输出参数，组的 oguid 值

rww_status: 输出参数，守护进程状态

inst_ok: 输出参数，实例是否正常

istatus: 输出参数，实例的系统状态

imode: 输出参数，实例的模式

trstat: 输出参数，即时归档状态

mcstat: 输出参数，主库和备库之间的 mal 链路状态

pname: 输出参数，备库对应的主库实例名

pocnt: 输出参数，主库 open 次数，备库和主库的值相等

flsn: 输出参数，flsn 值

clsn: 输出参数，clsn 值

sslsn: 输出参数，sslsn 值

slsn: 输出参数，slsn 值

dseq: 输出参数，最后收到的、守护进程发送的报文序号

code: 输出参数，dseq 对应报文的执行结果

mon_dseq: 输出参数，最后收到的、监视器发送的报文序号
mon_code: 输出参数，mon_dseq 对应报文的执行结果

注意：该接口中所有的字符串输出参数，长度必须大于 16，以免发生溢出。

返回值：

0：执行成功
<0：执行失败

19. 获取指定序号的实例信息

函数原型：

```
DMRWM_RETURN
dmrwm_get_nth_inst_info(
    mhandle         mhdl,
    muint4          nth,
    mchar*          inst_name,
    mchar*          group_name,
    muint4*         oguid,
    mchar*          rww_status,
    mchar*          inst_ok,
    mchar*          istatus,
    mchar*          imode,
    mchar*          trstat,
    mchar*          mcstat,
    mchar*          pname,
    muint4*         pocnt,
    muint8*         flsn,
    muint8*         clsn,
    muint8*         sslsn,
    muint8*         slsn,
    muint4*         dseq,
    mint4*          code,
    muint4*         mon_dseq,
    mint4*          mon_code
);
```

功能说明：

获取第 nth 个实例的站点信息，由 dmrwm_get_group_info 或 dmrwm_get_n_inst 可以获取到组中的实例个数 n_inst，nth 取值范围为 0 到 n_inst-1。

参数说明：

mhdl: 输入参数，监视器操作句柄
nth: 输入参数，取第 nth 个实例的站点信息
inst_name: 输出参数，实例名称
group_name: 输出参数，实例所在的组名

oguid: 输出参数，组的 oguid 值
rww_status: 输出参数，守护进程状态
inst_ok: 输出参数，实例是否正常
istatus: 输出参数，实例的系统状态
imode: 输出参数，实例的模式
trstat: 输出参数，即时归档状态
mcstat: 输出参数，主库和备库之间的 mal 链路状态
pname: 输出参数，备库对应的主库实例名
pocnt: 输出参数，主库 open 次数，备库和主库的值相等
flsn: 输出参数，flsn 值
clsn: 输出参数，clsn 值
sslsn: 输出参数，sslsn 值
slsn: 输出参数，slsn 值
dseq: 输出参数，最后收到的、守护进程发送的报文序号
code: 输出参数，dseq 对应报文的执行结果
mon_dseq: 输出参数，最后收到的、监视器发送的报文序号
mon_code: 输出参数，mon_dseq 对应报文的执行结果

注意：该接口中所有的字符串输出参数，长度必须大于 16，以免发生溢出。

返回值：

0：执行成功
<0：执行失败

20. 打开所有守护进程的监控

函数原型：

```
DMRWM_RETURN
dmrwm_startup_all(
    mhandle             mhdl
);
```

功能说明：

打开所有守护进程的监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

>=0：所有守护进程的监控都打开成功。执行前已经有守护进程的监控处于打开状态时，返回值为 100。
<0：执行失败

21. 打开指定的守护进程监控

函数原型：

```
DMRWM_RETURN
```

```
dmrwm_startup_single(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

打开指定实例的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的守护实例名称

返回值：

>=0：执行成功，执行前守护进程的监控已经处于打开状态时，返回值为 100

<0：执行失败

22. 关闭所有守护进程的监控

函数原型：

```
DMRWM_RETURN
dmrwm_stop_all(
    mhandle         mhdl
);
```

功能说明：

关闭所有守护进程的监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 SHUTDOWN 状态。

如果需要正常的退出实例或者保持实例为 MOUNT 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

>=0：所有守护进程的监控都关闭成功。执行前已经有守护进程的监控处于关闭状态时，返回值为 100。

<0：执行失败。

23. 关闭指定的守护进程监控

函数原型：

```
DMRWM_RETURN
dmrwm_stop_single(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

关闭指定实例守护进程的监控功能，关闭后守护进程处于 SHUTDOWN 状态。

如果需要正常的退出实例或者保持实例为 MOUNT 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的守护实例名称

返回值：

>=0：执行成功，执行前守护进程的监控已经处于关闭状态时，返回值为 100。

<0：执行失败

24. 启动所有实例

函数原型：

```
DMRWM_RETURN
dmrwm_inst_startup_all(
    mhandle             mhdl
);
```

功能说明：

启动所有实例，在 dmrww.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

>=0：所有实例都启动成功。执行前已经有实例已经处于活动状态时，返回值为 100。

<0：执行失败

25. 启动指定实例

函数原型：

```
DMRWM_RETURN
dmrwm_inst_startup_single(
    mhandle             mhdl,
    mchar*              inst_name
);
```

功能说明：

启动指定的实例，在 dmrww.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

mhdle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的实例名称

返回值：

>=0：执行成功，执行前实例已经处于活动状态时，返回值为 100。

<0：执行失败

26. 退出所有实例

函数原型：

```
DMRWM_RETURN
dmrwm_inst_exit_all(
    mhandle             mhdl
);
```

功能说明：

关闭守护进程和退出所有实例。

此接口执行的操作为先关闭所有守护进程的监控功能，然后通知主库实例退出，再通知所有备库实例退出。

注意守护进程只是状态切换为 SHUTDOWN，并没有退出，另外执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

>=0：所有实例退出成功。执行前实例已经退出时返回值为 100。

<0：执行失败

27. 退出指定实例

函数原型：

```
DMRWM_RETURN
dmrwm_inst_exit_single(
    mhandle             mhdl,
    mchar*              inst_name
);
```

功能说明：

退出指定的实例，退出的顺序为先关闭该实例守护进程的监控功能，再通知实例正常退出，以防实例退出后再被守护进程重新拉起，如果退出失败则重新启动守护进程的监控功能。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的实例名称

返回值：

>=0：执行成功，执行前实例已经退出时，返回值为 100。

<0：执行失败

28. OPEN 所有实例

函数原型：

```
DMRWM_RETURN
dmrwm_open_all(
    mhandle         mhdl
);
```

功能说明：

OPEN 所有实例，执行顺序为先 OPEN 所有备库，最后 OPEN 主库，目前守护进程监控功能打开的情况下，会自动执行 OPEN 操作，一般不需要再调用此接口执行。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

>=0：所有实例 OPEN 成功。执行前已经有实例已经处于 OPEN 状态，返回值为 100。

<0：执行失败

29. OPEN 指定实例

函数原型：

```
DMRWM_RETURN
dmrwm_open_single(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

OPEN 指定的实例，正常情况下，守护进程的监控功能打开的情况下，会自动 OPEN 所监控的实例，一般不需要再调用此接口执行。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定需要 OPEN 的实例名

返回值：

>=0：执行成功，执行前实例已经处于 OPEN 状态时返回值为 100.

<0：执行失败

30. 切换主备库模式

函数原型：

```
DMRWM_RETURN
dmrwm_switchover(
    mhandle         mhdl,
```

```
    mchar*              inst_name
);
```

功能说明：

在系统主备库和守护进程均正常的情况下，调用此接口可以完成指定的备库和主库模式的互相切换。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

返回值：

0：执行成功

<0：执行失败

31. 获取可接管的备库个数

函数原型：

```
DMRWM_RETURN
dmrwm_get_choose_num(
    mhandle             mhdl,
    muint2*             n_choose
);
```

功能说明：

在主库出现故障无法恢复，需要选择一个符合条件的备库执行接管操作时，调用此接口可以获取到可接管的备库个数，需要和dmrwm_get_nth_choose_name配合使用，获取到可接管的备库实例名。如果个数为0，则不存在可接管备库。

参数说明：

mhdl: 输入参数，监视器操作句柄

n_choose: 输出参数，返回可接管备库的个数

返回值：

0：执行成功

<0：执行失败

32. 获取可接管的备库实例名

函数原型：

```
DMRWM_RETURN
dmrwm_get_nth_choose_name(
    mhandle             mhdl,
    muint2              nth,
    mchar*              name_out
);
```

功能说明：

获取第nth个可接管备库的实例名，由dmrwm_get_choose_num接口可以获取到可接管的

备库个数n_choose，nth取值范围为0到n_choose-1。

参数说明：

mhdl: 输入参数，监视器操作句柄

nth: 输入参数，指定获取第 nth 个可接管的备库实例名

name_out: 输出参数，返回可接管的备库实例名

返回值：

0：执行成功

<0：执行失败

33. 接管故障主库

函数原型：

```
DMRWM_RETURN
dmrwm_takeover_use(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

使用指定的备库实例接管故障主库。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

返回值：

0：执行成功

<0：执行失败

34. 强制接管故障主库

函数原型：

```
DMRWM_RETURN
dmrwm_takeover_force_use(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

使用指定的备库实例强制接管故障主库，不要求接管用的备库即时归档是有效状态。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

返回值：

0：执行成功

<0：执行失败

35. 恢复指定的实例

函数原型：

```
DMRWM_RETURN
dmrwm_recover(
    mhandle         mhdl,
    mchar*          inst_name
);
```

功能说明：

恢复指定的实例。

对于出现故障的备库，如果 dmrww.ini 中的 RWW_MODE 设置为 1，则备库的守护进程会自动执行恢复，不需要再调用此接口执行。

对于故障重启的主库，该接口适用于当前系统中已经有新主库，需要将故障主库恢复为备库重加入系统的情况。

注意：执行此操作需要先登录监视器。

参数说明：

mhdl: 输入参数，监视器操作句柄

inst_name: 输入参数，需要恢复的实例名

返回值：

0：执行成功

<0：执行失败

36. 退出登录监视器

函数原型：

```
DMRWM_RETURN
dmrwm_logout(
    mhandle         mhdl
);
```

功能说明：

退出登录监视器

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

37. 获取退出标记

函数原型：

```
mbool
dmrwm_get_exit_flag(
    mhandle             mhdl
);
```

功能说明：

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 TRUE 时可正常退出线程。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

TRUE: 允许线程退出

FALSE: 不允许线程退出，需要继续等待获取输出消息。

38. 设置消息退出事件

函数原型：

```
void
dmrwm_set_exit_event(
    mhandle             mhdl
);
```

功能说明：

如果上层应用程序中有单独的消息线程，在 dmrwm_get_exit_flag 接口获取到退出标记为 TRUE 时，需要调用此接口设置退出事件。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

无

39. 退出消息事件等待

函数原型：

```
void
dmrwm_msg_event_exit(
    mhandle             mhdl
);
```

功能说明：

使用等待事件方式获取监视器消息时，需要单独起一个线程，在程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

消息线程相关的接口有 dmrwm_msg_event_wait、dmrwm_get_exit_flag、dmrwm_get_exec_msg、dmrwm_set_exit_event 和 dmrwm_msg_event_exit，这几个接口需要配合使用。

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

无

40. 销毁监视器环境

函数原型：

```
DMRWM_RETURN
dmrwm_deinit(
    mhandle             mhdl
);
```

功能说明：

销毁监视器执行环境

参数说明：

mhdl: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

41. 释放操作句柄

函数原型：

```
DMRWM_RETURN
dmrwm_free_handle(
    mhandle             mhdl
);
```

功能说明：

释放分配到的监视器句柄

参数说明：

mhdl: 输入参数，监视器操作句柄。

返回值：

0：执行成功

<0：执行失败

1.3 JNI 接口说明

1. 分配操作句柄

函数原型：

```
long
dmrwm_alloc_handle()
```

功能说明：

分配监视器操作句柄，监视器的接口都通过此句柄调用执行。

参数说明：

无

返回值：

0：分配失败。

>0：分配成功，返回值为监视器操作句柄。

2. 初始化监视器环境

函数原型：

```
int
dmrwm_init(
    long      handle,
    byte[]    ini_path,
    int       ini_len,
    byte[]    log_path,
    int       log_len
);
```

功能说明：

初始化监视器的环境

参数说明：

handle: 输入参数，监视器操作句柄

ini_path: 输入参数，指定 dmrwm.ini 的绝对路径

ini_len: 输入参数，ini_path 的字节长度

log_path: 输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则不生成日志文件

log_len: 输入参数，log_path 的字节长度

返回值：

0: 执行成功

<0: 执行失败

3. 等待消息事件

函数原型：

```
void
dmrwm_msg_event_wait(
    long      handle
);
```

功能说明：

监视器检测到守护进程或实例状态发生变化，或者接口命令执行过程中，产生中间输出消息时会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等待事件发生后，可通过接口 dmrwm_get_exec_msg 去读取中间结果。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

无

4. 获取输出信息

函数原型：

```
int
dmrwm_get_exec_msg(
    long                handle,
    byte[]              buf_msg,
    int                 buf_len,
    DmRwmValueData msg_len_out,
    DmRwmValueData get_flag
);
```

功能说明：

获取输出信息，和 dmrwm_msg_event_wait 配合使用。

参数说明：

handle:

输入参数，监视器操作句柄

buf_msg：

输出参数，保存获取到的输出消息，注意 buf_msg 缓存长度需要大于 2048，以免长度不够导致消息被截断。

buf_len:

输入参数，指定 buf_msg 可写入的最大长度，建议 buf_msg 缓存长度及 buf_len 输入值大于 2048，避免消息被截断。

msg_len_out:

输出参数，写消息成功后，实际写入的消息长度保存到 msg_len_out 中。

get_flag:

输出参数，是否继续读取消息，TRUE 表示还有消息可读，FALSE 表示已全部读完。

返回值：

0：执行成功

<0：执行失败

5. 设置自动显示间隔

函数原型：

```
int
dmrwm_set_show_interval(
    long                handle,
    int                 show_interval
```

```
);
```

功能说明：

监视器根据指定的 show_interval 值，每隔 show_interval 个时间间隔，自动输出各实例的状态信息给用户，需要使用 dmrwm_get_exec_msg 接口获取到输出消息，show_interval 单位为秒，默认值为 0。

参数说明：

handle: 输入参数，监视器操作句柄

show_interval: 输入参数，设置定时输出状态信息的时间间隔，单位为秒，取值为 0，或 5~3600，取值为 0 表示取消自动输出，默认值为 0。

返回值：

<0：执行失败

其他值：执行成功

6. 获取自动显示间隔

函数原型：

```
int
dmrwm_get_show_interval(
    long                handle
);
```

功能说明：

获取当前设置的显示时间间隔，单位为秒。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

<0：获取失败

其他值：当前设置的自动显示时间间隔

7. 设置自动写入日志文件间隔

函数原型：

```
int
dmrwm_set_log_interval(
    long                handle,
    int                 log_interval
);
```

功能说明：

监视器按照设置的 log_interval 值，每隔 log_interval 个时间间隔，自动将各实例的状态信息写入到日志文件，log_interval 单位为秒，默认值为 1。

参数说明：

handle: 输入参数，监视器操作句柄

log_interval: 输入参数，设置定时写入状态信息到日志文件的时间间隔，单位为秒，取值为 0，1 或 5~3600。

取值为 0 表示不写入任何信息到日志文件。

取值为 1 表示只写入普通的状态变化日志，例如收到新的信息或者检测到状态变化等。

取值为 5~3600 范围内的数值时，除了生成普通日志信息外，会按照设置的间隔定期写入站点状态信息到日志文件中。

返回值：

<0：执行失败

其他值：执行成功

8. 获取自动写入日志文件间隔

函数原型：

```
int
dmrwm_get_log_interval(
    long                handle
);
```

功能说明：

获取当前设置的自动写入日志的时间间隔，单位为秒。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

9. 设置单个日志文件大小

函数原型：

```
int
dmrwm_set_log_file_size(
    long                handle,
    int                 log_file_size
);
```

功能说明：

调用此接口，可以设置监视器单个的日志文件大小，达到最大值后，会自动生成并切换到新的日志文件中，如果达到最大的日志总空间限制，会自动删除创建时间最早的日志文件，日志文件命名方式为“dmrwm_端口号_年月日时分秒.log”，例如“dmrwm_55681_20150418230523.log”。

参数说明：

handle: 输入参数，监视器操作句柄

log_file_size: 输入参数，设置单个日志文件大小，单位为 M，取值范围为 16~2048，默认值为 64。

返回值：

<0：执行失败

其他值：执行成功

10. 获取单个日志文件大小

函数原型：

```
int
dmrwm_get_log_file_size(
    long                handle
);
```

功能说明：

获取当前设置的单个日志文件大小，单位为 M。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

11. 设置日志总空间大小

函数原型：

```
int
dmrwm_set_log_space_limit(
    long                handle,
    int                 log_space_limit
);
```

功能说明：

调用此接口，可以设置监视器的日志总空间大小，达到设置的总空间大小后，会自动删除创建时间最早的日志文件。

参数说明：

handle: 输入参数，监视器操作句柄

log_space_limit: 输入参数，设置日志总空间大小，单位为 M，取值为 0 或 256~4096，默认值为 0，表示没有空间限制。

返回值：

<0：执行失败

其他值：执行成功

12. 获取日志总空间大小

函数原型：

```
int
dmrwm_get_log_space_limit(
    long                handle
);
```

功能说明：

获取当前设置的日志总空间大小，单位为 M。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

<0：执行失败。

其他值：执行成功。

13. 登录监视器

函数原型：

```
int
dmrwm_login(
    long      handle,
    byte[]    username,
    int       user_len,
    byte[]    password,
    int       pwd_len
);
```

功能说明：

登录监视器，这里的登录口令和服务器的登录口令是一致的，且必须有 DBA 权限。

校验登录信息时由守护进程转发给服务器校验，并返回校验结果给监视器，因此需要保证系统中至少有一个活动实例，否则无法校验成功，

参数说明：

handle: 输入参数，监视器操作句柄

username: 输入参数，登录用户名

user_len: 输入参数，username 的字节长度

password: 输入参数，登录密码

pwd_len: 输入参数，password 的字节长度

返回值：

0：执行成功

<0：执行失败

14. 获取监控组信息

函数原型：

```
DmRwmGroup
dmrwm_get_group_info(
    long      handle
);
```

功能说明：

获取监控组信息

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

返回 DmRwmGroup 对象，为 NULL 则执行失败

15. 获取系统当前状态

函数原型：

```
int
dmrwm_tip(
    long        handle
);
```

功能说明：

调用此接口返回当前系统的状态信息，通过 dmrwm_get_exec_msg 接口可以获取返回消息，如果有日志文件也可以通过读日志文件获取。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

16. 获取所有的实例名

函数原型：

```
String[]
dmrwm_get_all_inst_name(
    long        handle
);
```

功能说明：

获取监控组中所有实例的实例名。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

返回实例名数组，为 NULL 则获取失败。

17. 根据实例名获取站点信息

函数原型：

```
DmRwmData
dmrwm_get_inst_info_by_name(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

根据指定的实例名，获取对应的站点信息

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定要获取的实例名称

len: 输入参数，inst_name 的字节长度

返回值：

返回 DmRwmData 对象，为 NULL 则获取失败

18. 获取指定序号的实例信息

函数原型：

```
DmRwmData
dmrwm_get_nth_inst_info(
    long        handle,
    int         nth
);
```

功能说明：

获取第 nth 个实例的站点信息，由 dmrwm_get_group_info 可以获取到组中的实例个数 n_inst，nth 取值范围为 0 到 n_inst-1。

参数说明：

handle: 输入参数，监视器操作句柄

nth: 输入参数，取第 nth 个实例的站点信息

返回值：

返回 DmRwmData 对象，为 NULL 则获取失败

19. 获取所有实例信息

函数原型：

```
DmRwmData[]
dmrwm_get_inst_list(
    long        handle
);
```

功能说明：

获取所有实例信息。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

返回实例信息数组，为 NULL 则获取失败

20. 打开所有守护进程的监控

函数原型：

```
int
dmrwm_startup_all(
    long        handle
);
```

功能说明：

打开所有守护进程的监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

>=0：所有守护进程的监控都打开成功。执行前已经有守护进程的监控处于打开状态时，返回值为 100。

<0：执行失败

21. 打开指定的守护进程监控

函数原型：

```
int
dmrwm_startup_single(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

打开指定实例的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的守护实例名称

len: 输入参数，inst_name 的字节长度

返回值：

>=0：执行成功，执行前守护进程的监控已经处于打开状态时，返回值为 100

<0：执行失败

22. 关闭所有守护进程的监控

函数原型：

```
int
dmrwm_stop_all(
    long        handle
);
```

功能说明：

关闭所有守护进程的监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 SHUTDOWN 状态。

如果需要正常的退出实例或者保持实例为 MOUNT 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

>=0：所有守护进程的监控都关闭成功。执行前已经有守护进程的监控处于关闭状态时，返回值为 100。

<0：执行失败

23. 关闭指定的守护进程的监控

函数原型：

```
int
dmrwm_stop_single(
    long      handle,
    byte[]    inst_name,
    int       len
);
```

功能说明：

关闭指定实例守护进程的监控功能，关闭后守护进程处于 SHUTDOWN 状态。

如果需要正常的退出实例或者保持实例为 MOUNT 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的守护实例名称

len: 输入参数，inst_name 的字节长度

返回值：

>=0：执行成功，执行前守护进程的监控已经处于关闭状态时，返回值为 100。

<0：执行失败

24. 启动所有实例

函数原型：

```
int
dmrwm_inst_startup_all(
    long      handle
);
```

功能说明：

启动所有实例，在 dmrww.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

>=0：所有实例都启动成功。执行前已经有实例已经处于活动状态时，返回值为 100。

<0：执行失败

25. 启动指定实例

函数原型：

```
int
dmrwm_inst_startup_single(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

启动指定的实例，在 dmrww.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的实例名称

len: 输入参数，inst_name 的字节长度

返回值：

>=0：执行成功，执行前实例已经处于活动状态时，返回值为 100。

<0：执行失败

26. 退出所有实例

函数原型：

```
int
dmrwm_inst_exit_all(
    long        handle
);
```

功能说明：

关闭守护进程和退出所有实例。

此接口执行的操作为先关闭所有守护进程的监控功能，然后通知主库实例退出，再通知所有备库实例退出。

注意守护进程只是状态切换为 SHUTDOWN，并没有退出，另外执行此操作需要先登录

监视器。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

>=0：所有实例退出成功。执行前实例已经退出时返回值为 100。

<0：执行失败

27. 退出指定实例

函数原型：

```
int
dmrwm_inst_exit_single(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

退出指定的实例，退出的顺序为先关闭该实例守护进程的监控功能，再通知实例正常退出，以防实例退出后再被守护进程重新拉起，如果退出失败则重新启动守护进程的监控功能。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的实例名称

len: 输入参数，inst_name 的字节长度

返回值：

>=0：执行成功，执行前实例已经退出时，返回值为 100。

<0：执行失败

28. OPEN 所有实例

函数原型：

```
int
dmrwm_open_all(
    long        handle
);
```

功能说明：

OPEN 所有实例，执行顺序为先 OPEN 所有备库，最后 OPEN 主库，目前守护进程监控功能打开的情况下，会自动执行 OPEN 操作，一般不需要再调用此接口执行。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

>=0：所有实例 OPEN 成功。执行前已经有实例已经处于 OPEN 状态，返回值为 100。

<0：执行失败

29. OPEN 指定实例

函数原型：

```
int
dmrwm_open_single(
    long      handle,
    byte[]    inst_name,
    int       len
);
```

功能说明：

OPEN 指定的实例，正常情况下，守护进程的监控功能打开的情况下，会自动 OPEN 所监控的实例，一般不需要再调用此接口执行。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定需要 OPEN 的实例名

len: 输入参数，指定 inst_name 的字节长度

返回值：

>=0：执行成功，执行前实例已经处于 OPEN 状态时返回值为 100.

<0：执行失败

30. 切换主备库模式

函数原型：

```
int
dmrwm_switchover(
    long      handle,
    byte[]    inst_name,
    int       len
);
```

功能说明：

在系统主备库和守护进程均正常的情况下，调用此接口可以完成指定的备库和主库模式的互相切换。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

len: 输入参数，inst_name 的字节长度

返回值：

0：执行成功

<0：执行失败

31. 获取可接管的备库实例名

函数原型：

```
String[]
dmrwm_get_all_choose_name(
    long        handle
);
```

功能说明：

在主库出现故障无法恢复，需要选择一个符合条件的备库执行接管操作时，调用此接口可以获取到所有可接管的备库实例名。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

返回备库实例名数组，为 NULL 则获取失败或者不存在可接管的备库。

32. 接管故障主库

函数原型：

```
int
dmrwm_takeover_use(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

使用指定的备库实例接管故障主库。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

len: 输入参数，inst_name 的字节长度

返回值：

0：执行成功

<0：执行失败

33. 强制接管故障主库

函数原型：

```
int
dmrwm_takeover_use(
    long        handle,
    byte[]      inst_name,
```

```
    int         len
);
```

功能说明：

使用指定的备库实例强制接管故障主库，不要求接管用的备库即时归档是有效状态。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，指定的备库实例名

len: 输入参数，inst_name 的字节长度

返回值：

0：执行成功

<0：执行失败

34. 恢复指定的实例

函数原型：

```
int
dmrwm_recover(
    long        handle,
    byte[]      inst_name,
    int         len
);
```

功能说明：

恢复指定的实例。

对于出现故障的备库，如果 dmrww.ini 中的 RWW_MODE 设置为 1，则备库的守护进程会自动执行恢复，不需要再调用此接口执行。

对于故障重启的主库，该接口适用于当前系统中已经有新主库，需要将故障主库恢复为备库重加入系统的情况。

注意：执行此操作需要先登录监视器。

参数说明：

handle: 输入参数，监视器操作句柄

inst_name: 输入参数，需要恢复的实例名

len: 输入参数，inst_name 的字节长度

返回值：

0：执行成功

<0：执行失败

35. 退出登录监视器

函数原型：

```
int
dmrwm_logout(
```

```
    long        handle
);
```

功能说明：

退出登录监视器

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

36. 获取退出标记

函数原型：

```
boolean
dmrwm_get_exit_flag(
    long        handle
);
```

功能说明：

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 TRUE 时可正常退出线程。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

true: 允许线程退出

false: 不允许线程退出，需要继续等待获取输出消息。

37. 设置消息退出事件

函数原型：

```
void
dmrwm_set_exit_event(
    long        handle
);
```

功能说明：

如果上层应用程序中有单独的消息线程，在 dmrwm_get_exit_flag 接口获取到退出标记为 true 时，需要调用此接口设置退出事件。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

无

38. 退出消息事件等待

函数原型：

```
void
dmrwm_msg_event_exit(
    long        handle
);
```

功能说明：

使用等待事件方式获取监视器消息时，需要单独起一个线程，在程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

无

39. 销毁监视器环境

函数原型：

```
int
dmrwm_deinit(
    long        handle
);
```

功能说明：

销毁监视器执行环境

参数说明：

handle: 输入参数，监视器操作句柄

返回值：

0：执行成功

<0：执行失败

40. 释放操作句柄

函数原型：

```
int
dmrwm_free_handle(
    long        handle
);
```

功能说明：

释放分配到的监视器句柄

参数说明：

handle: 输入参数，监视器操作句柄。

返回值：

0：执行成功
<0：执行失败

1.4 Java 相关类说明

1. DmRwmGroup 类

DmRwmGroup 类的有效成员变量说明如下，获取和设置这些变量可以使用相应的 get 和 set 方法，其他一些成员变量暂不支持，另外该类需要定义在 com.dameng.console.dmrwm 包路径下。

group_name: String，监控组名
comm_type: String，监视器的通信方式
primary_name: String，组中的主库实例名
mon_port: int，监视器的通信端口
oguid: int，组的 oguid 值
n_site: int，组中的实例个数

2. DmRwmData 类

DmRwmData 类的有效成员变量说明如下，获取和设置这些变量可以使用相应的 get 和 set 方法，其他一些成员变量暂不支持，另外该类需要定义在 com.dameng.console.dmrwm 包路径下。

group_name: String， 监控组名
rwwstat_str: String，守护进程状态
inst_ok_str: String，实例状态是否正常
iname_str: String，实例名称
istatus_str: String，实例的系统状态
imode_str: String，实例的模式
trstat_str: String，实例的即时归档状态
pname_str: String，备库实例对应的主库实例名，主库实例该字段为空
open_cnt: long，主库 open 次数，备库和主库的值相等
flsn: long，实例的 flsn 值
clsn: long，实例的 clsn 值
sslsn: long，实例的 sslsn 值
slsn: long，实例的 slsn 值
dseq: long，最后收到的、守护进程发送的报文序号
code: long，dseq 对应的执行结果

3. DmRwmValueData 类

DmRwmValueData 类的有效成员变量说明如下，其他一些成员变量暂不支持，该类需

要定义在 com.dameng.console.dmrwm 包路径下，该类用于接口的输出参数类型，接口调用时直接对相关的成员变量赋值并传出。

type: byte，存放类型

value: int，存放 int 类型的值

obj: Object，存放对象类型的值

int64: long，存放 long 类型的值

bytes: byte[]，存放 byte[]类型值

flag: boolean，存放 boolean 类型值

注：dmrwm_get_exec_msg 接口中的输出值保存在 value 成员中。

1.5 C 编程示例

这里给出 C 程序的编程示例，运行环境为 VS2010，通过配置项目属性，使用 dmrwm_dll.lib 隐式加载 dll 的方式。

注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

1. test_dmrwm.c

c 文件 test_dmrwm.c:

```
#include "dmrwm_dll.h"
#include "stdio.h"
#include "stdlib.h"
#include "windows.h"
#include "process.h"

/* 消息线程 */
DWORD
WINAPI
dmrwm_get_msg_thread(
    mhandle        handle
)
{
    mschar              buf_msg[4097];
    muint4              msg_len_out;
    muint4              get_flag;
    DMRWM_RETURN       code;

    while (TRUE)
    {
        /* 等待消息事件 */
        dmrwm_msg_event_wait(handle);
```

```
        /* 获取退出标记 */
        if (dmrwm_get_exit_flag(handle) == TRUE)
        {
            break;
        }

        do
        {
            /* 获取并打印消息 */
            code = dmrwm_get_exec_msg(handle, buf_msg, 4097, &msg_len_out, &get_flag);
            if (code >=0 && msg_len_out > 0)
            {
                fprintf(stdout, "%s", buf_msg);
            }

        } while (get_flag); /* 判断是否还有未读消息 */
    }

    /* 设置退出事件 */
    dmrwm_set_exit_event(handle);

    return 0;
}

/* 清理环境 */
void
dmrwm_clear_env(
    mhandle        handle
)
{
    /* 通知并等待 dmrwm_get_msg_thread 消息线程退出 */
    dmrwm_msg_event_exit(handle);

    /* 销毁监视器环境 */
    dmrwm_deinit(handle);

    /* 释放句柄 */
    dmrwm_free_handle(handle);
}

int
main()
{
```

```
DMRWM_RETURN        ret;
mhandle             handle;
HANDLE              thread_handle;

/*  分配操作句柄  */
ret = dmrwm_alloc_handle(&handle);
if (ret < 0)
{
    fprintf(stdout, "dmrwm alloc handle failed!\n");
    return ret;
}

/*  创建消息线程  */
thread_handle = (HANDLE)_beginthreadex(NULL, 0, dmrwm_get_msg_thread, handle, 0, NULL);
if (thread_handle == NULL)
{
    ret = GetLastError();
    fprintf(stderr, "_beginthreadex error! desc:%s, code:%d\n", strerror(ret), ret);

    return ret;
}

/*  初始化监视器环境  */
ret = dmrwm_init(handle, "D:\\dmrwm\\dmrwm.ini", "D:\\dmrwm");
if (ret < 0)
{
    dmrwm_clear_env(handle);
    fprintf(stdout, "dmrwm init failed!\n");

    return ret;
}

/* sleep 2s  等待 watch 消息  */
Sleep(2000);

/*  登录监视器  */
ret = dmrwm_login(handle, "SYSDBA", "SYSDBA");
if (ret < 0)
{
    fprintf(stdout, "dmrwm login failed!\n");
}

/*  执行其他命令  */
/*...*/
```

```
    /* 退出登录 */
    dmrwm_logout(handle);

    /* 清理环境 */
    dmrwm_clear_env(handle);

    system("pause");

    return 0;
}
```

1.6 Java 编程示例

调用 JNI 接口时，需要定义 DmRwmGroup，DmRwmData， DmRwmValueData 类，相关的类成员说明可参考 1.4 小节，类定义需要放在 com.dameng.console.dmrwm 包路径下，相关文件说明如下，注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

1. DmRwmGroup.java

DmRwmGroup 类定义文件：

```
package com.dameng.console.dmrwm;

public class DmRwmGroup {
    private String group_name;
    private String comm_type;
    private String primary_name;
    private int mon_port;
    private int oguid;
    private int n_site;

    public String getGroup_name() {
        return group_name;
    }
    public void setGroup_name(String group_name) {
        this.group_name = group_name;
    }
    public String getComm_type() {
        return comm_type;
    }
    public void setComm_type(String comm_type) {
        this.comm_type = comm_type;
```

```
	}
	public String getPrimary_name() {
		return primary_name;
	}
	public void setPrimary_name(String primary_name) {
		this.primary_name = primary_name;
	}
	public int getMon_port() {
		return mon_port;
	}
	public void setMon_port(int mon_port) {
		this.mon_port = mon_port;
	}
	public int getOguid() {
		return oguid;
	}
	public void setOguid(int oguid) {
		this.oguid = oguid;
	}
	public int getN_site() {
		return n_site;
	}
	public void setN_site(int n_site) {
		this.n_site = n_site;
	}
}
```

2. DmRwmData.java

DmRwmData 类定义文件：

```
package com.dameng.console.dmrwm;

public class DmRwmData {
	private String group_name;
	private String rwwstat_str;
	private String inst_ok_str;
	private String iname_str;
	private String istatus_str;
	private String imode_str;
	private String trstat_str;
	private String pname_str;
	private long open_cnt;
	private long flsn;
```

```
private long clsn;
private long sslsn;
private long slsn;
private long dseq;
private long code;

public String getGroup_name() {
	return group_name;
}
public void setGroup_name(String group_name) {
	this.group_name = group_name;
}
public String getRwwstat_str() {
	return rwwstat_str;
}
public void setRwwstat_str(String rwwstat_str) {
	this.rwwstat_str = rwwstat_str;
}
public String getInst_ok_str() {
	return inst_ok_str;
}
public void setInst_ok_str(String inst_ok_str) {
	this.inst_ok_str = inst_ok_str;
}
public String getIname_str() {
	return iname_str;
}
public void setIname_str(String iname_str) {
	this.iname_str = iname_str;
}
public String getIstatus_str() {
	return istatus_str;
}
public void setIstatus_str(String istatus_str) {
	this.istatus_str = istatus_str;
}
public String getImode_str() {
	return imode_str;
}
public void setImode_str(String imode_str) {
	this.imode_str = imode_str;
}
public String getTrstat_str() {
	return trstat_str;
```

```
}
public void setTrstat_str(String trstat_str) {
	this.trstat_str = trstat_str;
}
public String getPname_str() {
	return pname_str;
}
public void setPname_str(String pname_str) {
	this.pname_str = pname_str;
}
public long getOpen_cnt() {
	return open_cnt;
}
public void setOpen_cnt(long open_cnt) {
	this.open_cnt = open_cnt;
}
public long getFlsn() {
	return flsn;
}
public void setFlsn(long flsn) {
	this.flsn = flsn;
}
public long getClsn() {
	return clsn;
}
public void setClsn(long clsn) {
	this.clsn = clsn;
}
public long getSslsn() {
	return sslsn;
}
public void setSslsn(long sslsn) {
	this.sslsn = sslsn;
}
public long getSlsn() {
	return slsn;
}
public void setSlsn(long slsn) {
	this.slsn = slsn;
}
public long getDseq() {
	return dseq;
}
public void setDseq(long dseq) {
```

```
            this.dseq = dseq;
        }
        public long getCode() {
            return code;
        }
        public void setCode(long code) {
            this.code = code;
        }
}
```

3. DmRwmValueData.java

DmRwmValueData 类定义文件：

```
package com.dameng.console.dmrwm;

public class DmRwmValueData {
        public byte type;
        public int value;
        public Object obj;
        public long int64;
        public byte[] bytes;
        public boolean flag;

        public byte getType()
        {
            return type;
        }

        public void setType(byte type)
        {
            this.type = type;
        }

        public int getValue()
        {
            return value;
        }

        public void setValue(int value)
        {
            this.value = value;
        }
```

```
    public Object getObj()
    {
        return obj;
    }

    public void setObj(Object obj)
    {
        this.obj = obj;
    }

    public long getInt64()
    {
        return int64;
    }

    public void setInt64(long int64)
    {
        this.int64 = int64;
    }

    public byte[] getBytes()
    {
        return bytes;
    }

    public void setBytes(byte[] bytes)
    {
        this.bytes = bytes;
    }

    public boolean isFlag()
    {
        return flag;
    }

    public void setFlag(boolean flag)
    {
        this.flag = flag;
    }
}
```

4. DmRwmDLL.java

该文件中定义相关 JNI 方法及加载 DLL 依赖库：

```
package com.dameng.console.dmrwm;

public class DmRwmDLL {

    private long handle = 0;

    //JNI 方法
    private static native int dmrwm_init(long handle, byte[] ini_path, int ini_len, byte[] log_path, int log_len);

    private static native int dmrwm_deinit(long handle);

    private static native int dmrwm_open_all(long handle);

    private static native int dmrwm_open_single(long handle, byte[] inst_name, int len);

    private static native int dmrwm_recover(long handle, byte[] inst_name, int len);

    private static native int dmrwm_switchover(long handle, byte[] inst_name, int len);

    private static native int dmrwm_takeover_use(long handle, byte[] inst_name, int len);

    private static native int dmrwm_startup_all(long handle);

    private static native int dmrwm_stop_all(long handle);

    private static native int dmrwm_startup_single(long handle, byte[] inst_name, int len);

    private static native int dmrwm_stop_single(long handle, byte[] inst_name, int len);

    private static native int dmrwm_inst_startup_all(long handle);

    private static native int dmrwm_inst_exit_all(long handle);

    private static native int dmrwm_inst_startup_single(long handle, byte[] inst_name, int len);

    private static native int dmrwm_inst_exit_single(long handle, byte[] inst_name, int len);

    private static native String[] dmrwm_get_all_inst_name(long handle);

    private static native String[] dmrwm_get_all_choose_name(long handle);
```

```
    private static native DmRwmData[] dmrwm_get_inst_list(long handle);

    private static native int dmrwm_login(long handle, byte[] username, int user_len, byte[] password, int pwd_len);

    private static native int dmrwm_logout(long handle);

    private static native void dmrwm_msg_event_wait(long handle);

    private static native void dmrwm_msg_event_exit(long handle);

    private static native int dmrwm_get_exec_msg(long handle, byte[] buf_msg, int buf_len, DmRwmValueData msg_len_out, DmRwmValueData get_flag);

    private static native DmRwmData dmrwm_get_inst_info_by_name(long handle, byte[] inst_name, int len);

    private static native DmRwmData dmrwm_get_nth_inst_info(long handle, int nth);

    private static native DmRwmGroup dmrwm_get_group_info(long handle);

    private static native int dmrwm_tip(long handle);

    private static native long dmrwm_alloc_handle();

    private static native int dmrwm_free_handle(long handle);

    private static native boolean dmrwm_get_exit_flag(long handle);

    private static native void dmrwm_set_exit_event(long handle);


    //加载 DLL
    static
    {
         System.loadLibrary("dmutl");
        System.loadLibrary("dmelog");
        System.loadLibrary("dmcvt");
        System.loadLibrary("dmos");
        System.loadLibrary("dmmsg");
        System.loadLibrary("dmmem");
        System.loadLibrary("dmstrt");
        System.loadLibrary("dmcpr");
        System.loadLibrary("dmcfg");
```

```
            System.loadLibrary("dmcyt");
            System.loadLibrary("dmcomm");
            System.loadLibrary("dmcalc");
            System.loadLibrary("dmuthr");
            System.loadLibrary("dmsys");
            System.loadLibrary("dmmout");

            //windows 环境
            System.loadLibrary("dmrwm_dll");
        }

        //自定义方法，这里只给出部分，可以自行扩充
        public int rwm_init(byte[] ini_path, int ini_len, byte[] log_path, int log_len)
        {
            return dmrwm_init(handle, ini_path, ini_len, log_path, log_len);
        }

        public int rwm_deinit()
        {
            return dmrwm_deinit(handle);
        }

        public int rwm_login(byte[] username, int user_len, byte[] password, int pwd_len)
        {
            return dmrwm_login(handle, username, user_len, password, pwd_len);
        }

        public int rwm_logout()
        {
            return dmrwm_logout(handle);
        }

        public void rwm_msg_event_wait()
        {
            dmrwm_msg_event_wait(handle);
        }

        public void rwm_msg_event_exit()
        {
            dmrwm_msg_event_exit(handle);
        }

        public int rwm_get_exec_msg(byte[] buf_msg, int buf_len, DmRwmValueData msg_len_out,
DmRwmValueData get_flag)
```

```
    {
        return dmrwm_get_exec_msg(handle, buf_msg, buf_len, msg_len_out, get_flag);
    }

    public boolean rwm_alloc_handle()
    {
        this.handle = dmrwm_alloc_handle();
        if (this.handle == 0)
            return false;

        return true;
    }

    public int rwm_free_handle()
    {
        return dmrwm_free_handle(this.handle);
    }

    public boolean rwm_get_exit_flag()
    {
        return dmrwm_get_exit_flag(this.handle);
    }

    public void rwm_set_exit_event()
    {
        dmrwm_set_exit_event(this.handle);
    }
}
```

5. DmRwmTest.java

程序入口文件，实际调用 JNI 接口：

```
package com.dameng.console.dmrwm;

import java.io.UnsupportedEncodingException;

public class DmRwmTest {
    public DmRwmDLL monDll = null;
    public Thread msgThread = null;

    public byte[] exec_msg = new byte[4097];
    public DmRwmValueData exec_msg_len = new DmRwmValueData();
    public DmRwmValueData get_flag = new DmRwmValueData();
```

```
public String ini_path = "D:\\dmrwm\\dmrwm.ini";
public String log_path = "D:\\dmrwm\\log";
public String username = "SYSDBA";
public String password = "SYSDBA";

//消息线程
public void startThread()
{
        msgThread = new Thread(new Runnable()
    {
        @Override
        public void run()
        {
           while (true)
              {
                 //等待消息事件
                 monDll.rwm_msg_event_wait();

                 //获取退出标记
                 if (monDll.rwm_get_exit_flag() == true)
                       break;

                    do
                    {
                       //获取并打印消息
                         int code = monDll.rwm_get_exec_msg(exec_msg, exec_msg.length,
                                  exec_msg_len, get_flag);

                         if (code == 0 && exec_msg_len.getValue() > 0)
                         {
                              try
                              {
                                   String msg = new String(exec_msg, 0, exec_msg_len.getValue(), "GBK");

                                   System.out.print(msg);
                              }
                              catch (UnsupportedEncodingException e)
                              {
                                   e.printStackTrace();
                              }
                         }
                    } while (get_flag.getValue() == 1); //是否有未读消息
              }
```

```
                //设置退出事件
                monDll.rwm_set_exit_event();
            }
        });

        //启动线程
        msgThread.start();
    }

    //退出线程
    public void endThread()
    {
        if (msgThread != null && msgThread.isAlive())
        {
            //通知并等待消息线程退出
            monDll.rwm_msg_event_exit();
        }
    }

    public void test() throws InterruptedException
    {
        boolean     ret1 = false;
        int         ret2 = 0;


        monDll = new DmRwmDLL();

        //分配句柄
        ret1 = monDll.rwm_alloc_handle();
        if (ret1 == false)
        {
            System.out.println("dmrwm alloc handle failed!");
            return;
        }
        else
        {
            System.out.println("dmrwm alloc handle success!");
        }

        //启动消息接收线程
        startThread();

        //初始化监视器
        ret2    =    monDll.rwm_init(ini_path.getBytes(),    ini_path.getBytes().length,    log_path.getBytes(),
```

```
log_path.getBytes().length);
            if (ret2 < 0)
            {
                System.out.println("dmrwm init failed!");

                //初始化失败，退出线程，销毁句柄
                endThread();
                monDll.rwm_free_handle();

                return;
            }
            else
            {
                System.out.println("dmrwm init success!");
            }

            //sleep 2s 接收 watch 消息
            Thread.sleep(2000);

            //登录监视器
            ret2 = monDll.rwm_login(username.getBytes(), username.getBytes().length, password.getBytes(),
password.getBytes().length);
            if (ret2 < 0)
                System.out.println("login failed, code:" + ret2);

            //退出登录
            monDll.rwm_logout();

            //退出消息线程
            endThread();

            //销毁监视器环境
            monDll.rwm_deinit();

            //销毁句柄
            monDll.rwm_free_handle();
        }

        /**
         * @param args
         * @throws InterruptedException
         */
        public static void main(String[] args) throws InterruptedException {
```

```
            DmRwmTest test = new DmRwmTest();
            test.test();
        }
}
```

1.7 DLL 依赖库

监视器 DLL 依赖了其他的一些动态链接库，因此编程时需要将达梦 bin 目录的以下库拷贝到和 dmrwm_dll.dll 或 libdmrwm.so 相同目录下，也可以在加载动态链接库时直接指定 bin 目录，就不需要再拷贝库文件：

序号	WINDOWS	非 WINDOWS（LINUX 等）
1	dmcalc.dll	libdmcalc.so
2	dmcfg.dll	libdmcfg.so
3	dmcomm.dll	libdmcomm.so
4	dmcpr.dll	libdmcpr.so
5	dmcvt.dll	libdmcvt.so
6	dmcyt.dll	libdmcyt.so
7	dmelog.dll	libdmelog.so
8	dmmem.dll	libdmmem.so
9	dmmout.dll	libdmmout.so
10	dmmsg.dll	libdmmsg.so
11	dmos.dll	libdmos.so
12	dmstrt.dll	libdmstrt.so
13	dmsys.dll	libdmsys.so
14	dmuthr.dll	libdmuthr.so
15	dmutl.dll	libdmutl.so

1.8 错误码汇编

代码	**解释**
-2	无效的句柄
-3	初始化日志失败
-4	初始化系统失败
-5	用户没有登录或者登录监视器失败
-6	读取 dmrwm.ini 失败（路径错误或配置错误或没有权限）
-503	内存不足
-810	系统未配置归档
-4702	非法的参数
-2501	用户名或密码错误，或用户没有 DBA 权限，或使用安全用户登录非安全版本的服务器

-6148	加密失败
-8167	用户名加密失败
-8168	密码加密失败
-8165	当前没有活动实例或守护进程状态不匹配，登录信息校验失败
-8174	用户名或密码为空或长度超长
-8014	用户自定义异常或者未知错误
-8006	无效的广播地址
-8010	获得本地信息失败
-8011	初始化接收端口失败
-8020	监视器还未收到守护进程消息
-8157	监视器未收到所有守护进程消息
-8038	有消息正在发送中,您的命令当前不能执行
-8042	实例故障, 操作终止
-8043	接收守护进程消息超时，操作终止
-8044	实例执行失败
-8045	指定实例名无效
-8046	实例的守护进程不是 STARTUP 状态
-8059	待恢复的实例是活动的主库
-8061	主库的守护进程 dmrww 为 ERROR 状态
-8062	实例的模式不匹配
-8063	待恢复的主库实例为非 MOUNT 状态
-8064	待恢复实例的 SLSN 大于主库
-8065	待恢复实例的即时归档是有效的，不能恢复
-8067	待恢复的主库不在控制文件中不能恢复
-8105	待恢复的主库 LSN 大于接管时备库的 LSN，不能恢复
-8052	实例的守护进程状态不匹配
-8071	监视器组不存在
-8106	Switchover 所用的实例已经是主库
-8074	Takeover 所用的实例已经是主库
-8075	主库实例正常，不符合 takeover 条件
-8078	实例状态不匹配
-8094	实例的守护进程状态不匹配
-8140	待接管备库即时归档不是有效状态
-8079	主库实例不是 PRIMARY&OPEN 状态，或主库 DMRWW 非 OPEN 状态
-8087	没有合适的即时备库可供选择
-8088	所有实例的守护进程已经处于 SHUTDOWN 状态
-8096	所有实例的守护进程已经处于 STARTUP 或 OPEN 状态
-8089	非 MOUNT 状态的主库不能 OPEN
-8090	实例故障
-8095	控制文件不存在或路径无效
-8097	系统中存在多个主库，命令不允许执行，需要用户干预
-8100	系统中存在多个 OPEN 状态的主库
-8110	接管使用的备库不在 CHOOSE 列表中

-8111	待切换备库即时归档不是有效状态
-8112	主库故障，操作终止
-8113	监视器操作冲突，终止操作
-8117	主库的守护进程 dmrww 不是 OPEN 状态
-8118	待恢复实例的守护进程 dmrww 不是 OPEN 状态
-8119	待恢复实例不是 OPEN 状态
-8120	接管使用实例的守护进程 dmrww 不是 OPEN 状态
-8121	实例为非 PRIMARY 或 STANDBY 模式
-8122	备库的 slsn 大于主库的 slsn
-8123	实例的守护进程 DMRWW 不是 OPEN 状态
-8124	实例状态为 ERROR
-8125	实例为非 OPEN 状态
-8126	实例的即时归档无效
-8127	实例的 OPEN_CNT 小于系统中其他 PRIMARY 实例，不允许手动 OPEN
-8128	设置主库的守护进程为恢复状态失败
-8137	检测到故障主库已恢复，重新修改实例为 STANDBY&OPEN 状态
-8185	实例的守护进程状态不匹配
-8155	通知启动实例失败
-8160	SHUTDOWN 所有实例失败
-8176	监视器尚未收到公钥信息，请稍候重试
-8180	实例的守护进程不是 STARTUP 或 OPEN 状态，或实例非 OK 状态，校验登录信息失败
-8181	待恢复实例的 CLSN 和 SSLSN 不一致，不允许恢复
-8188	接收守护进程消息超时
-8191	组不满足退出所有实例条件